



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The Protocol Stack

2.1 Introduction and upper layers

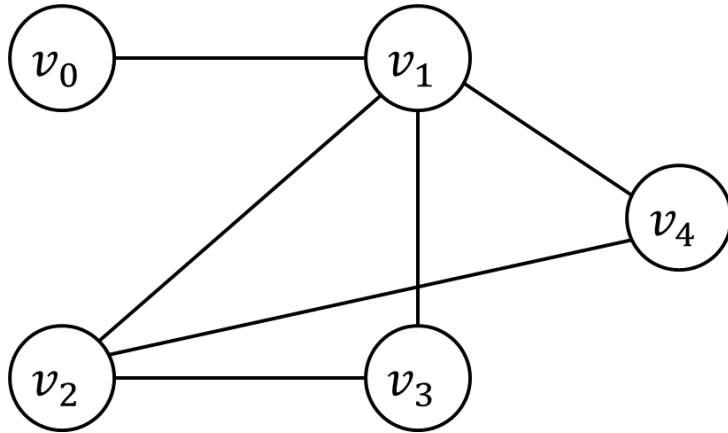
Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

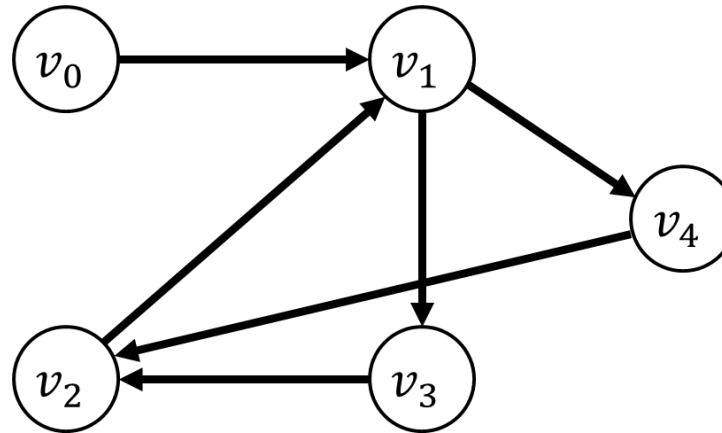
1. Introduction
2. The protocol stack (3 sessions)
 - **Introduction and upper layers**
 - Lower layers: network, link, physical
 - Other relevant protocols
 - Putting it all together: Wireshark
3. Impact on programming of on-line games
4. Other stories

Graph types

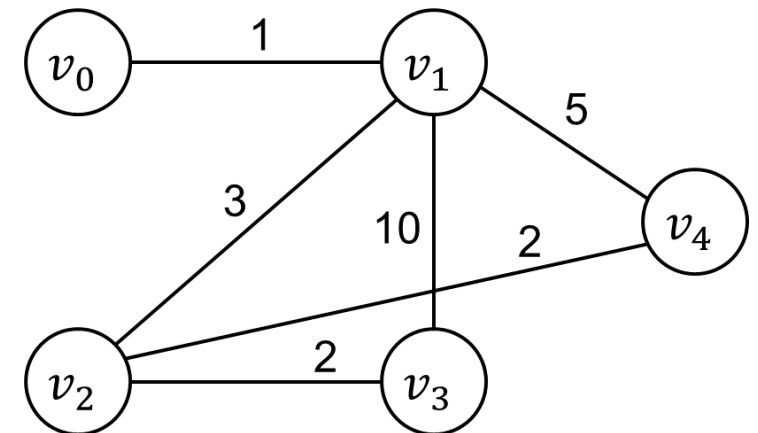
Normal graph
(bidirectional edges)



Directional graph
(unidirectional edges)



Weighted graph
(non-unit edges)



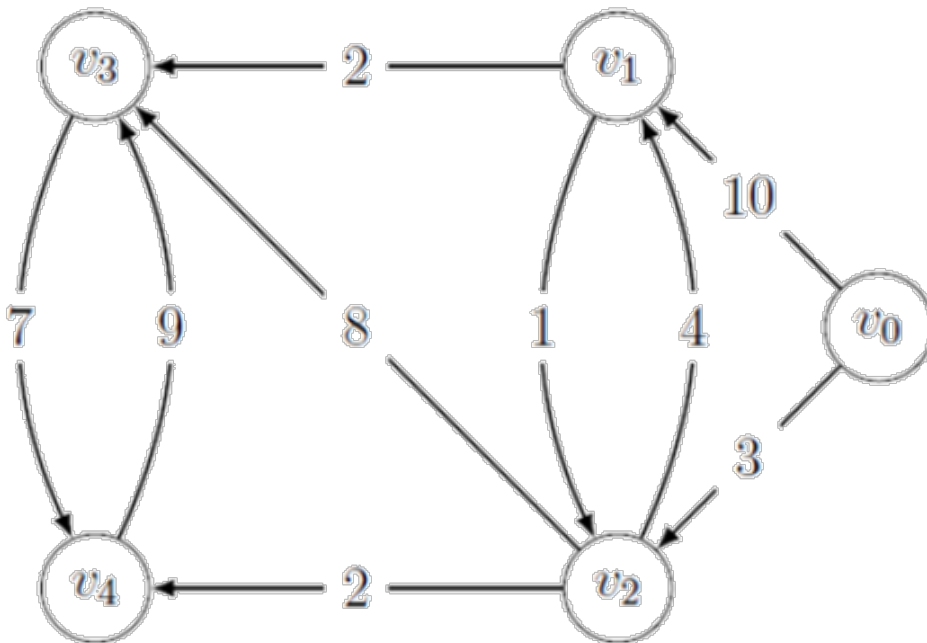
Which type of graph you think best represents a network like the Internet?



Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.



		Distàncies provisionals				
		v_0	v_1	v_2	v_3	v_4
Iteració	1	0	10	3	∞	∞
	2		7		11	5
	3		7		11	
	4				9	

Dijkstra's algorithm

Input: a weighted graph G and an initial node s .

$w(u, v) \in W$ is the weight of an edge between nodes u and v .

Output: minimum distance between s and the rest of nodes, minimum spanning tree

new dictionary $dist$

new dictionary Q

foreach node v in G **do**

$Q[v] = \infty$

$dist[v] = \infty$

end

$Q[s] = 0$

while Q is not empty **do**

$u = \min(Q)$

$dist[u] = Q[u]$

foreach node v adjacent to u **do**

if v is in Q **then**

if $Q[v] > Q[u] + w(u, v)$ **then**

$Q[v] = Q[u] + w(u, v)$

end

end

end

$delete(Q[u])$

end

$print(dist)$

Dijkstra's Algorithm

Developed by Dijkstra in 1956.

Finds the shortest path between two nodes in a weighted graph.

Widely used protocols such as OSPF or ISIS use Dijkstra's algorithm to determine the shortest path between any two nodes mostly backbone and large enterprise networks.

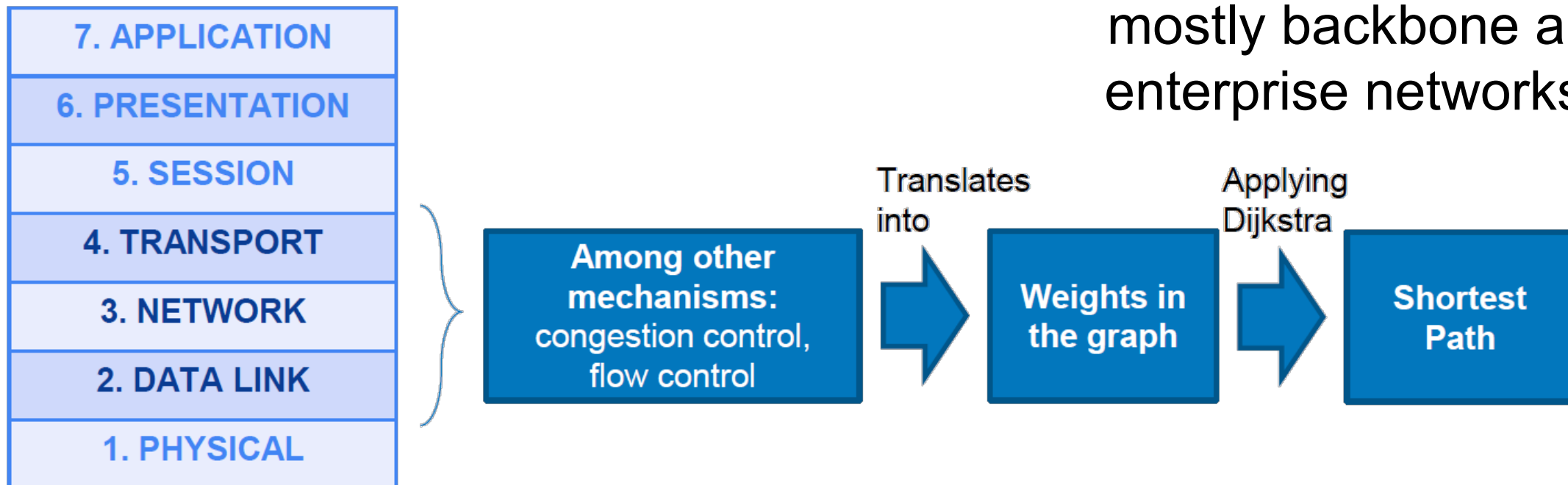
Implementations [\[edit \]](#)

- [Allied Telesis](#) implements OSPFv2 & OSPFv3 in Allied Ware Plus (AW+)
- [Arista Networks](#) implements OSPFv2 and OSPFv3
- [BIRD](#) implements both OSPFv2 and OSPFv3
- [Cisco IOS](#)
- [Cisco Meraki](#)
- [D-Link](#) implements OSPFv2 on Unified Services Router.
- [Dell's FTOS](#) implements OSPFv2 and OSPFv3
- [ExtremeXOS](#)
- [GNU Zebra](#), a GPL routing suite for [Unix-like](#) systems supporting OSPF
- [Juniper Junos](#)
- [Mikrotik RouterOS](#)
- [NetWare](#) implements OSPF in its Multi Protocol Routing module.
- [OpenBSD](#) includes [OpenSPFD](#), an OSPFv2 implementation.
- [Quagga](#), a fork of [GNU Zebra](#) for [Unix-like](#) systems
- [XORP](#), a routing suite implementing RFC2328 (OSPFv2) and RFC2740 (OSPFv3) for both IPv4 and IPv6
- [Windows NT 4.0 Server](#), [Windows 2000 Server](#) and [Windows Server 2003](#) implement OSPFv2 in the [Routing and Remote Access Service](#), although the functionality was removed in [Windows Server 2008](#).

Dijkstra's Algorithm

Developed by Dijkstra in 1956.
Finds the shortest path between two nodes in a weighted graph.

Widely used protocols such as OSPF or ISIS use Dijkstra's algorithm to determine the shortest path between any two nodes mostly backbone and large enterprise networks. **How?**





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

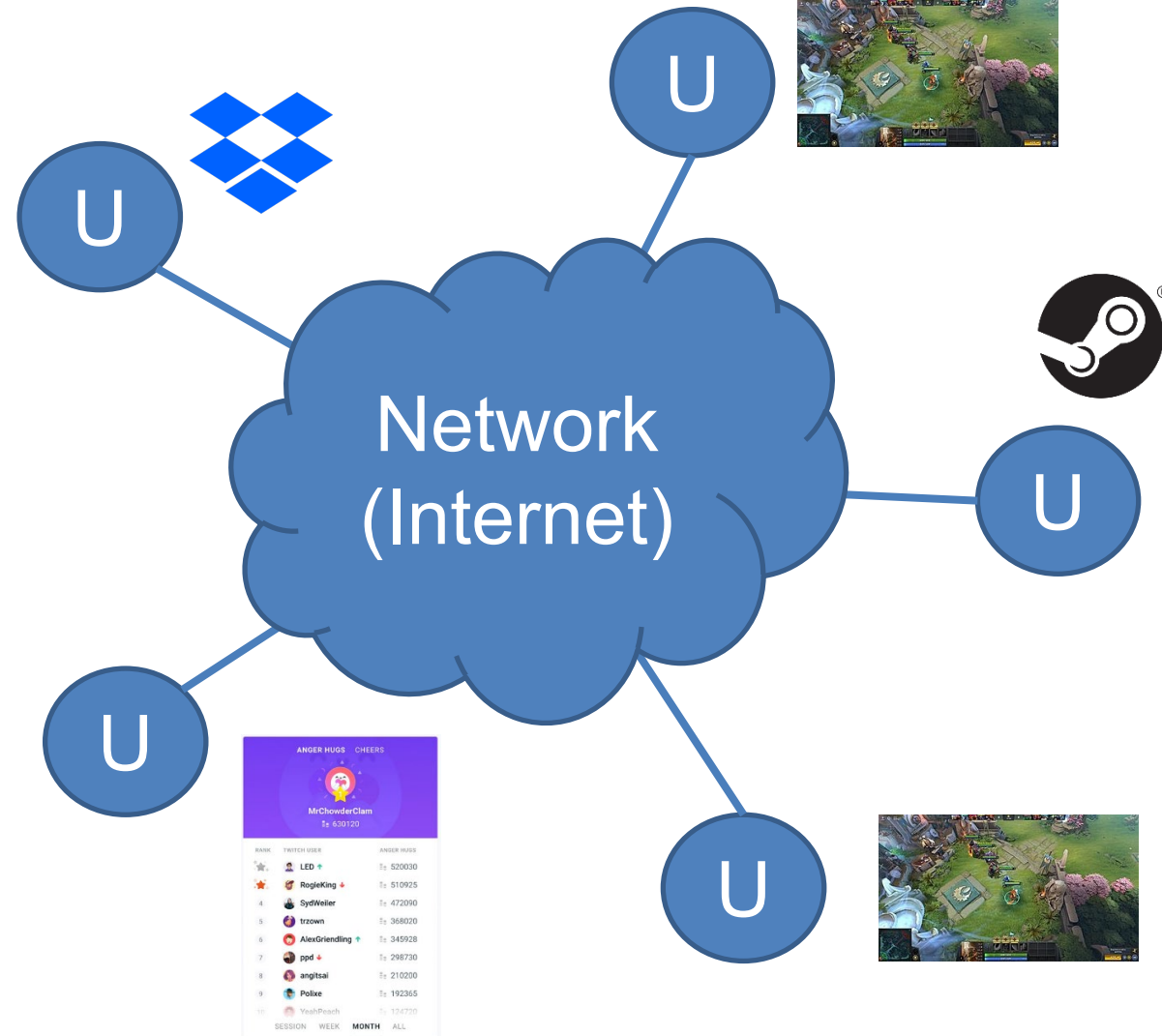
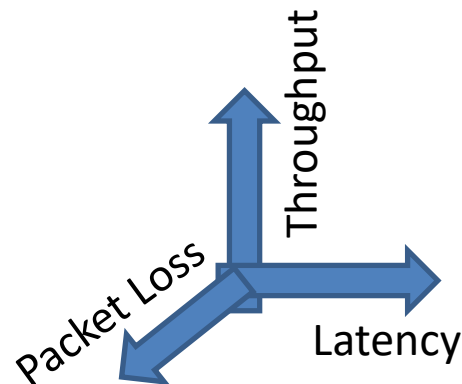
Centre de la Imatge i la Tecnologia Multimèdia

Introduction:

Origins and big picture

Internet: the network of networks

- Architectures
- Links
 - Wired or wireless
 - Different speeds or reliability
- Ever-changing topology
- Unpredictable traffic
- Heterogeneous needs
 - Transmit files
 - Streaming
 - Gaming
 - ...



Origins

- Networking (as any technical field) can be a jungle
- Need to put some order
- Enable interoperability
- Standardization organizations are in charge of putting this order



The OSI Reference Model



CITM

- The ISO proposed the **OSI Reference Model (Open System Interconnection)**, which is today's standard
- The model partitions the communications system into abstraction layers. Each layer has its own:
 - Scope
 - Functions
 - Data unit
- This allows designers to partition and focus



OSI Reference Model —The ISO Model of Architecture for Open Systems Interconnection,
Hubert Zimmermann, IEEE Transactions on Communications, vol. 28, no. 4, april 1980, pp. 425-432

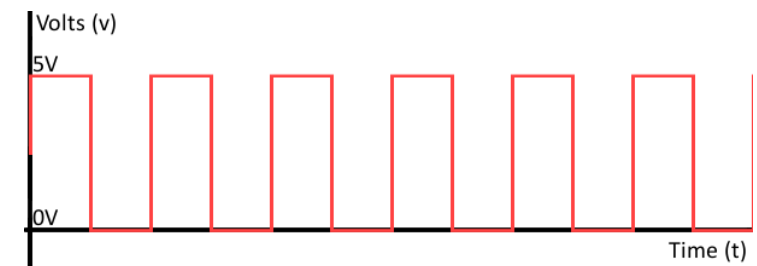
The OSI Layers

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

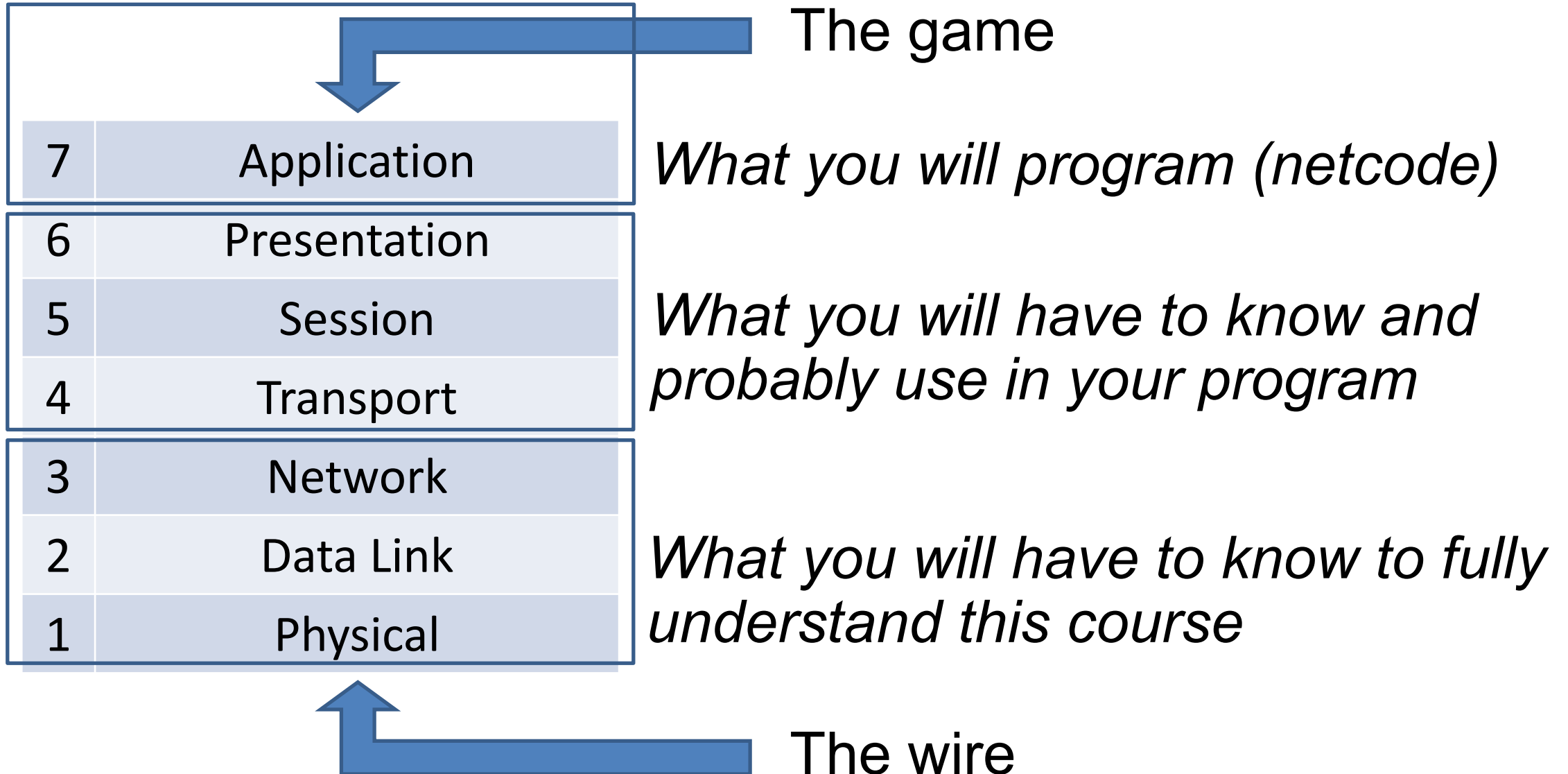
The wire



The OSI Layers



CITM

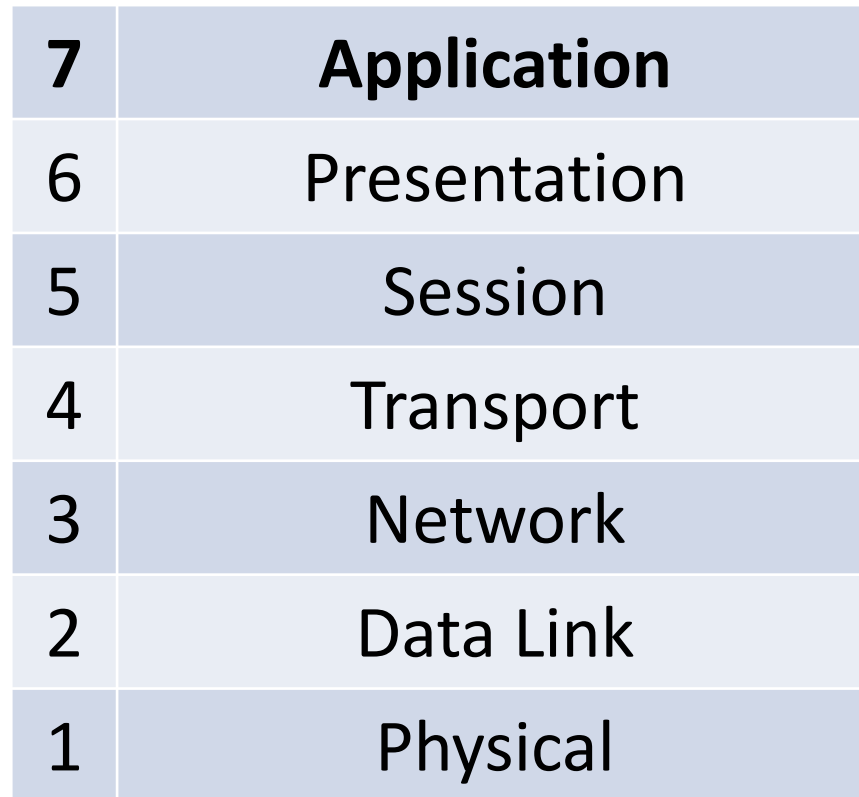


Application Layer



CITM

The game



A diagram of the seven layers of the OSI model, represented as a vertical stack of seven light blue rectangular boxes. Each box contains a number on the left and a layer name on the right. The layers are numbered 1 to 7 from bottom to top. A large blue arrow points from the top of the stack (Application layer) towards the text 'The game'. Another large blue arrow points from the bottom of the stack (Physical layer) towards the text 'The wire'.

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols and methods for communication between processes of an application
- **Scope:** application
- **Data unit:** application data
- **Example:** HTTP, DNS, *your game*

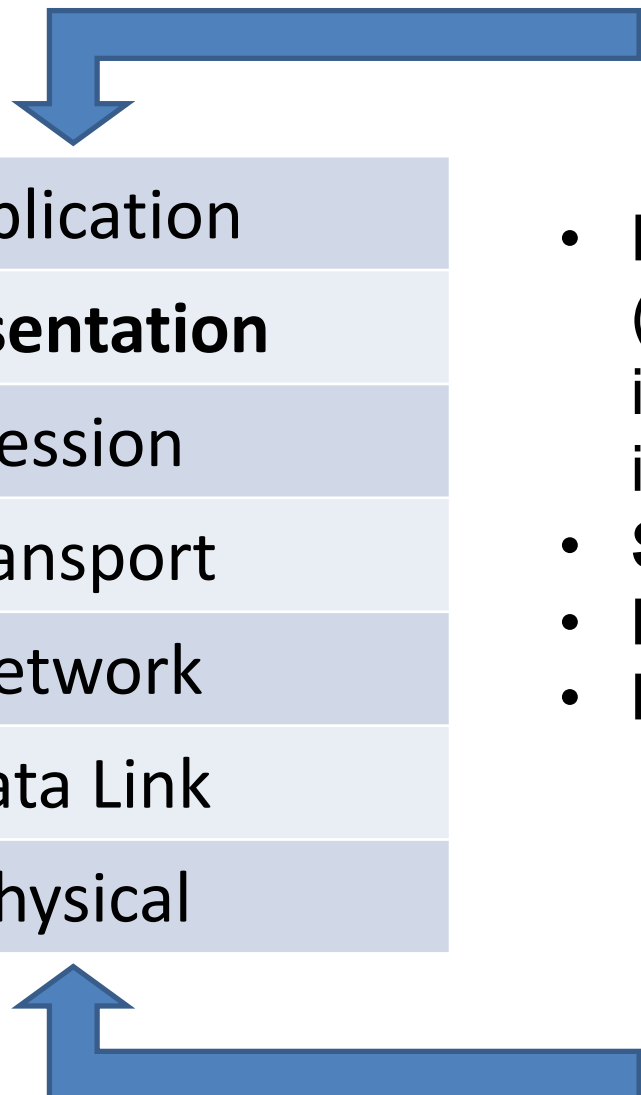
The wire

Presentation Layer



CITM

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for information representation (ASN.1). Syntaxis and semantics of information to be transmitted. It may include encryption
- **Scope:** display to display (for an app)
- **Data unit:** application data
- **Example:** Telnet, PNG

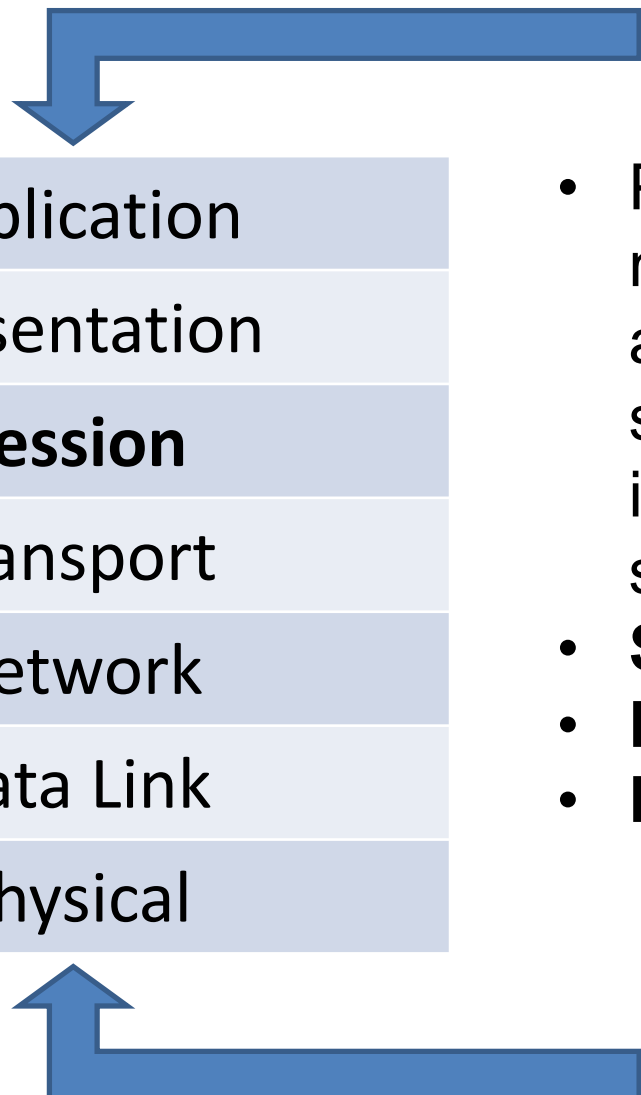
The wire

Session Layer



CITM

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for opening, closing, and managing a session between end user application processes (going beyond a single end to end connection). This includes authentication, tunneling, synchronization between flows, etc...
- **Scope:** users in a session
- **Data unit:** application data
- **Example:** AppleTalk

The wire

Transport Layer

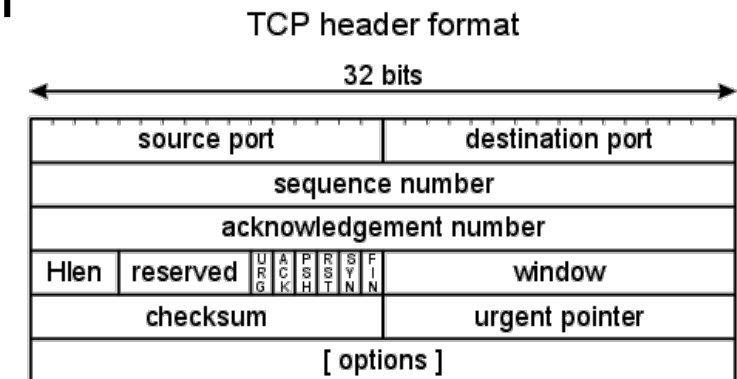
The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

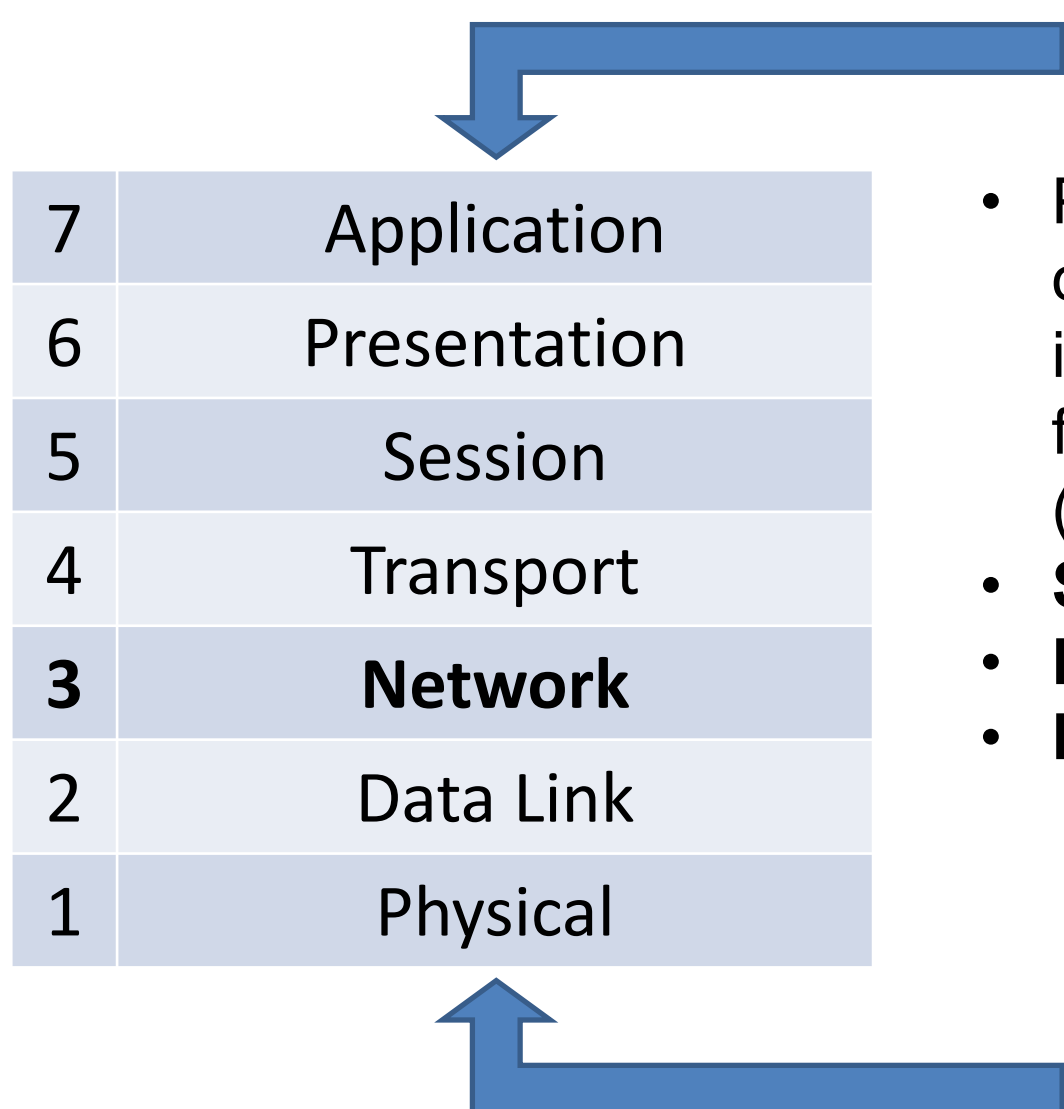
- Protocols to provide end to end connectivity and guarantees through sockets. Fragmentation and re assembly of messages, ordering. Also, congestion avoidance in end to end.
- **Scope:** two network end-points
- **Data unit:** datagram
- **Example:** TCP, UDP

The wire



Network Layer

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for routing packets through a set of nodes, from source to final destination. It includes methods for addressing and forwarding, as well as congestion control (change routes).
- **Scope:** two routers within the network
- **Data unit:** packet
- **Example:** IP

0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

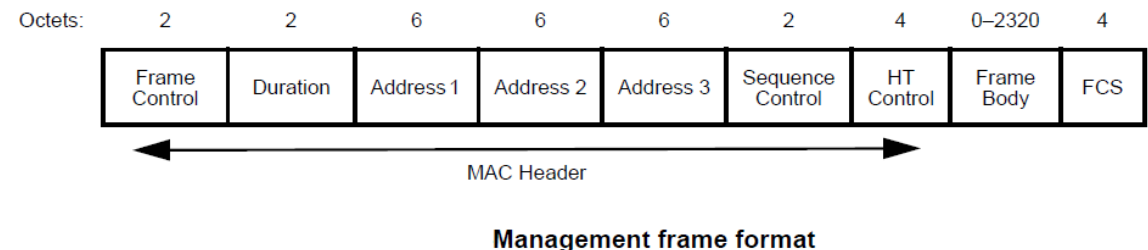
The wire

Data Link Layer

The game

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for point to point communication. Managing errors from the physical layer and acknowledging receptions (ACK). Medium access control, flow control.
- **Scope:** two machines connected by a wire
- **Data unit:** frame
- **Example:** 802.11 (WiFi)



The wire

Physical Layer

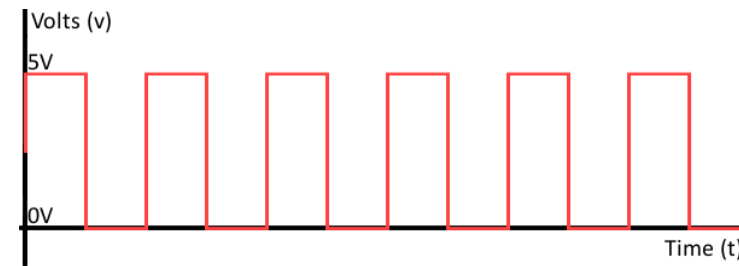


CITM

The game

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols that interface with the communications medium. Representation of raw bits and signal specifications
- **Scope:** within the comm. medium
- **Data unit:** symbol
- **Example:** X.21

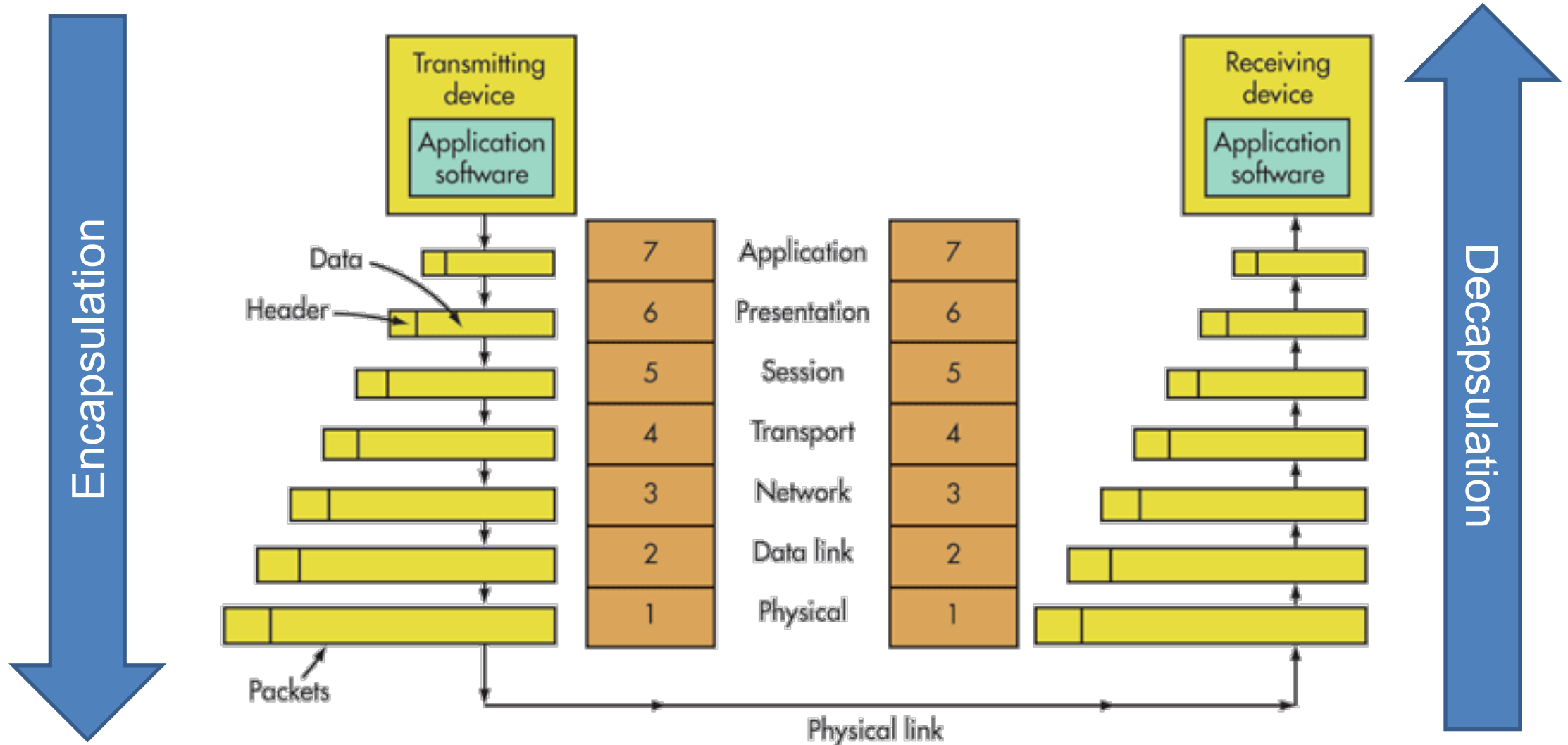


The wire

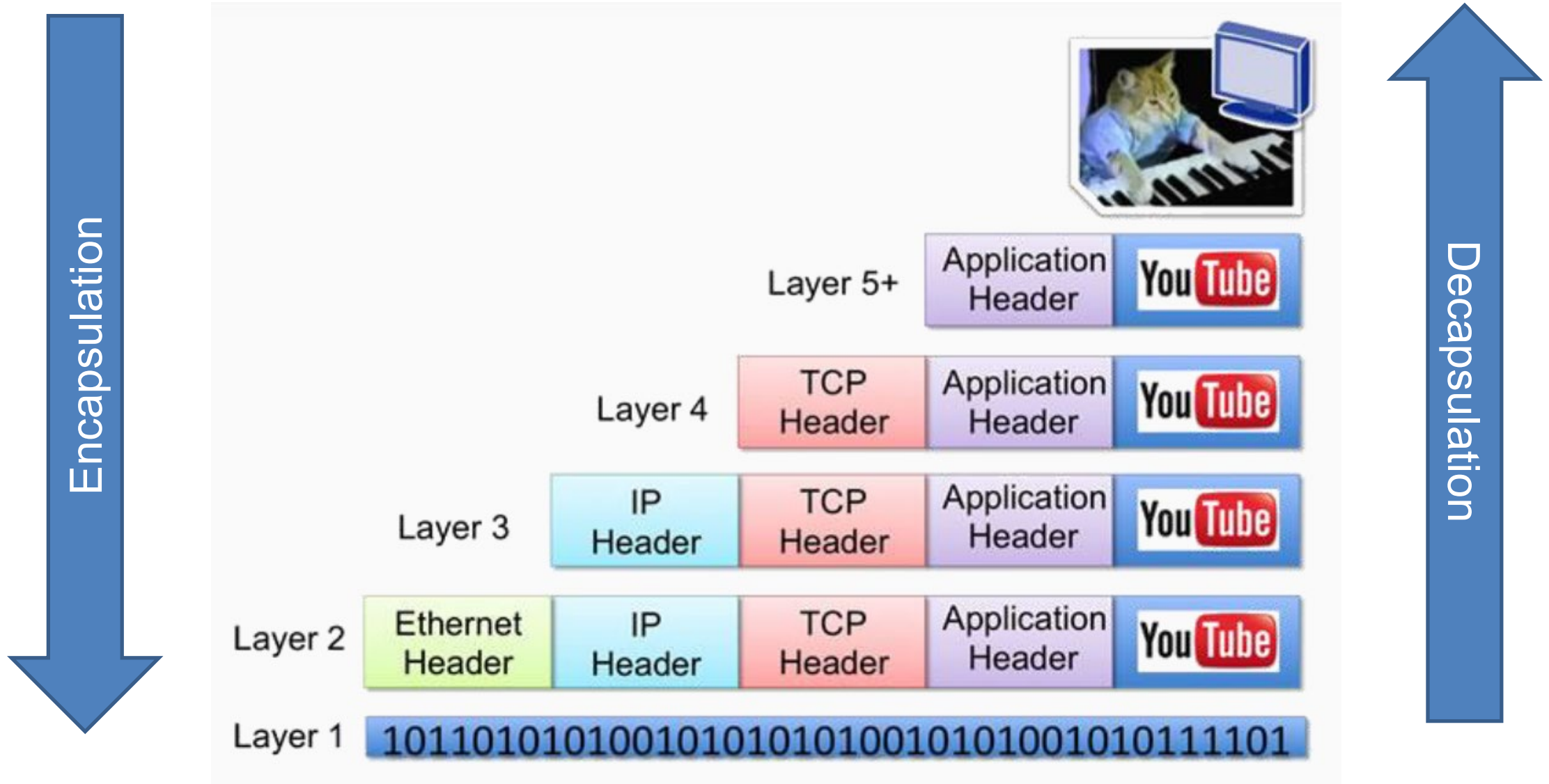
Encapsulation



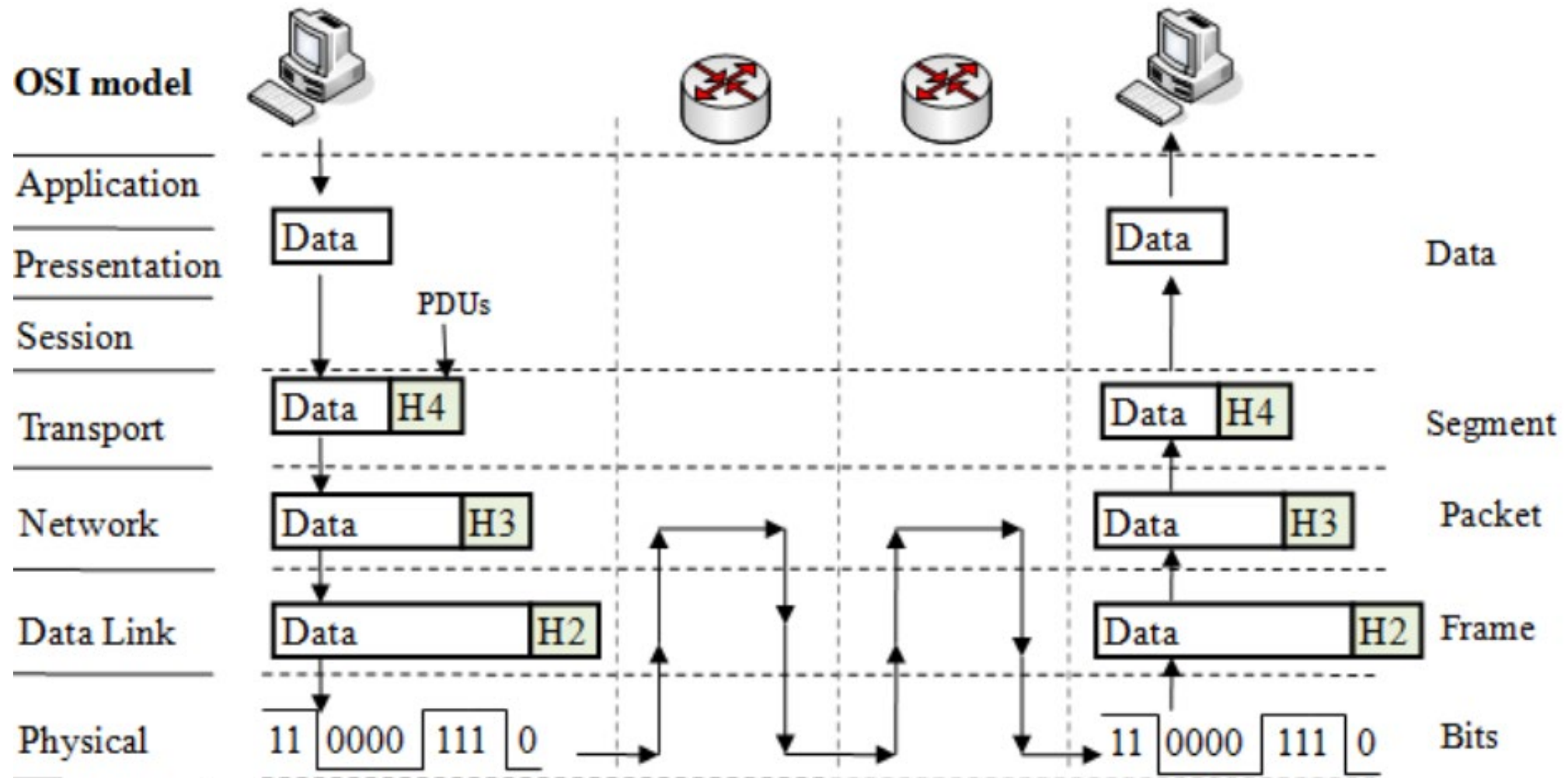
CITM



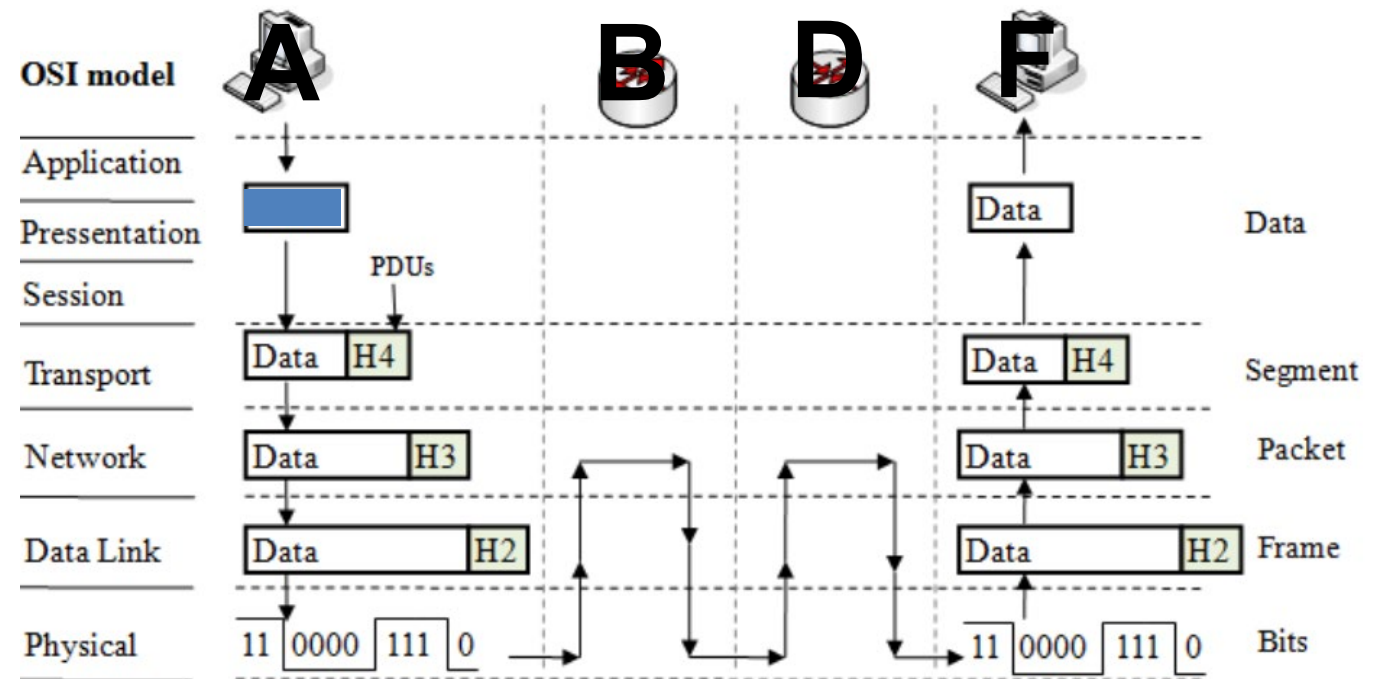
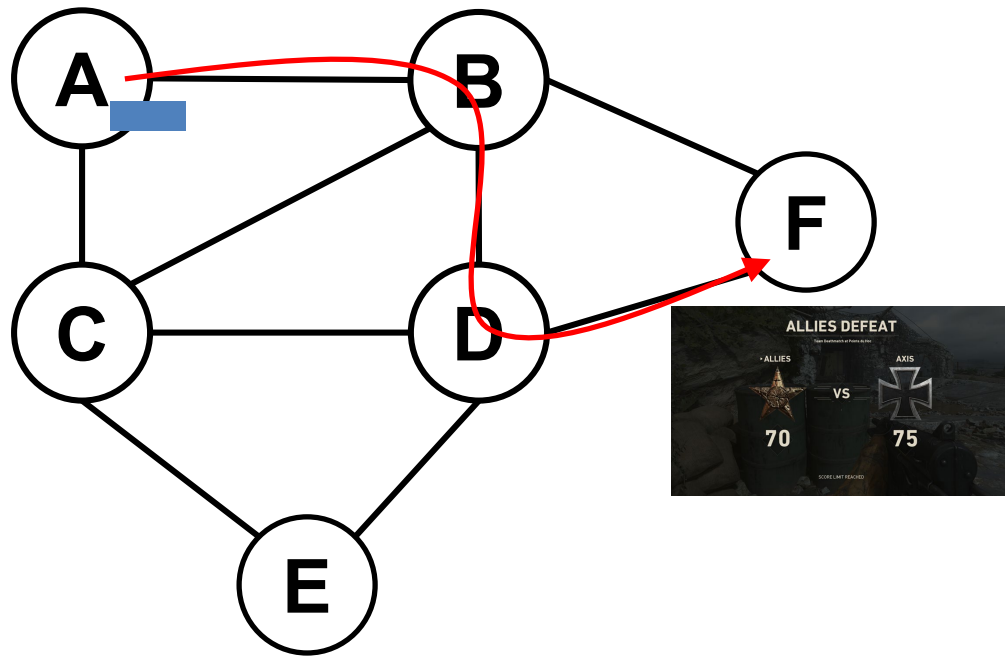
Encapsulation (example)



Life of a packet



Life of a packet (example)



Exercise – Protocol Overhead

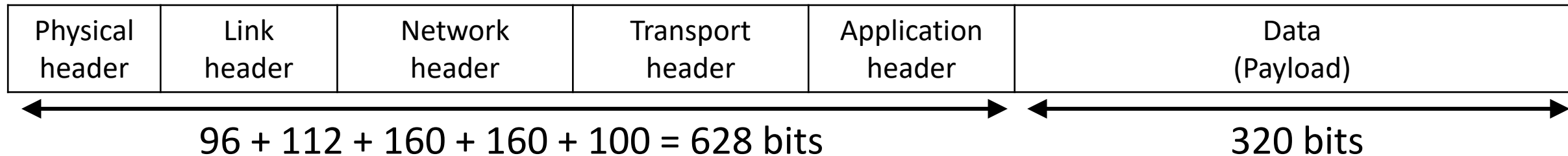
Our game needed to send 320 bits of data to transmit one movement of a player. Calculate the size of the transmitted packet after the process of encapsulation, the ratio of header to application data, and the throughput needed to transmit 60 moves per second. Assume the following header sizes:

- Application layer: 100 bits
- Transport layer: 160 bits
- Network layer: 160 bits
- Link layer: 112 bits
- Physical layer: 96 bits



Exercise – Protocol Overhead

Our game needed to send 320 bits of data to transmit one movement of a player. Calculate the size of the transmitted packet after the process of encapsulation, the ratio of header to application data, and the throughput needed to transmit 60 moves per second.

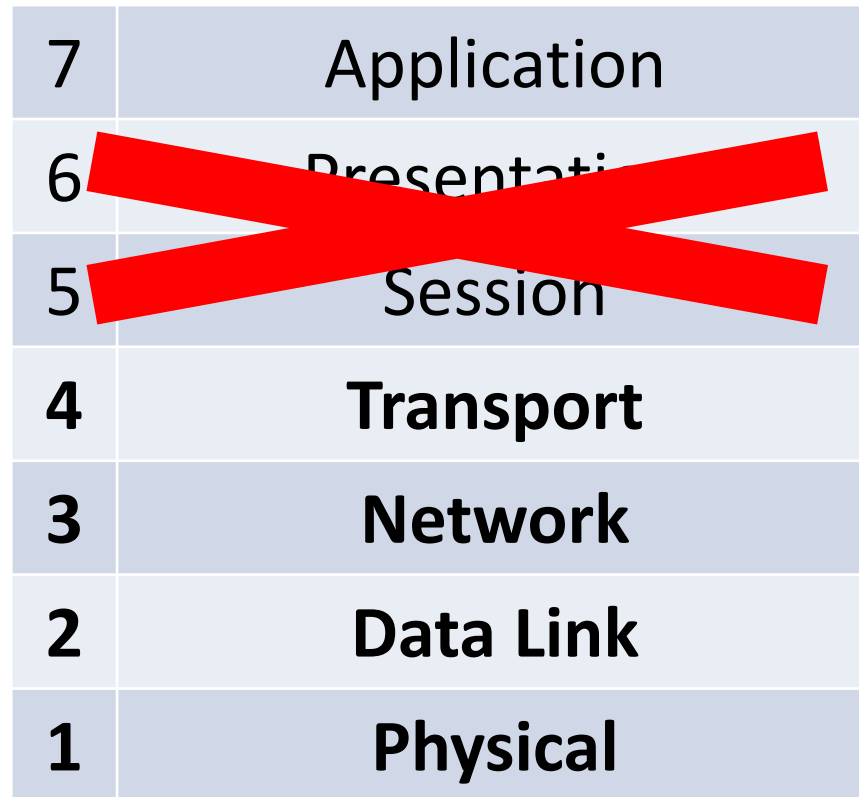


- **Total size:** $628 + 320 = 948 \text{ bits}$
- **Ratio:** $628 / 320 = 1.9625 = 196.25\%$
- **Throughput** at 60 FPS = $948 * 60 = 56.8 \text{ Kbps}$

Good or bad?
What should our
game do?

Application Layer

The game



- A couple of interesting application protocols will be explained later in the course, but the more interesting of it all will be **your game** (also kind of explained later in the course)
- Presentation and session protocols are not relevant to the course, but I encourage you to investigate (theory presentations)
- In this course we will mainly explain the four layers at the bottom of the stack.

The wire



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The Application Layer

Session 3: Data serialization

Application Layer



CITM

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

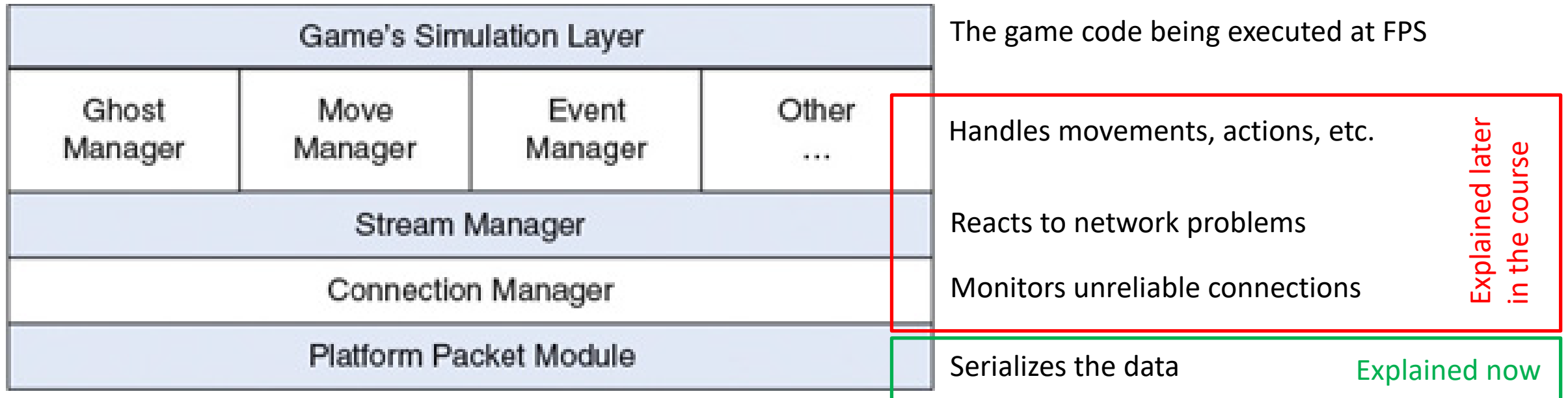
- Protocols and methods for communication between processes of an application
- **Scope:** application
- **Data unit:** application data
- **Example:** HTTP, DNS, your game

The wire



- For an on-line game, the application layer groups all the methods that allow all the remote processes (clients, server) to communicate, understand each other, and implement the illusion of coherence.
- This is typically also referred to as *netcode*.
- Netcode might include, for instance:
 - Handling and transmitting movements, actions, etc.
 - Monitoring the unreliable connections between clients and react to problems in the network
 - Serializing the data

Example: Starsiege Tribes

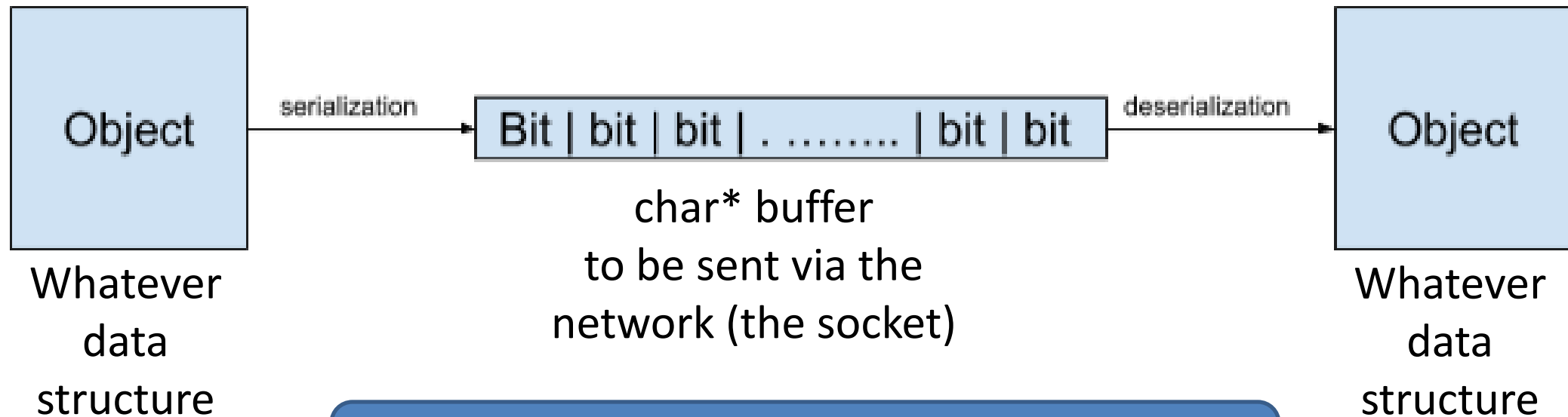


Object Serialization



CITM

- Anything that needs to be transmitted through a network need to be serialized. So we need to turn game objects and data structures into a series of bits



This is not safe unless both platforms have the exact same hardware and OS.

From the lab sessions

What is data serialization

What: Converting the data stored in memory into a linear sequence of bytes that can be transmitted over the network layer protocol (e.g. TCP/UDP).

Why: Its purpose is to ensure that both the sending and the receiving ends understand and interpret the data sent, in the same way.

However, this is not as simple as it may seem:

1. If we have virtual functions in a class, we cannot just serialize our pointer to that function.
 - In the destination, we would execute garbage or segfault
2. Similarly, if we have pointers in our structure, we cannot just pass the pointer to that data.
 - In the destination, we would write/read garbage or segfault

Object Serialization: The Problem (II)

3. If we define arrays of a given size, but only use part of it, we should only send the relevant parts to save bandwidth
4. For some classes such as std::vector, the internals of which are not always well-described or specified, it is simply not safe to just serialize it bit by bit.
5. On top of it, you may have endian compatibility issues.

In the end, what you need to do is to create a single buffer and safely serialize all data that can be also safely read at the other side.

Object Serialization: Alternatives

Alternative 1: JsonUtility

```
class testClass
```

Alternative 2: XmlSerializer

```
void deserializeXML()  
{  
    XmlSerializer serial  
    var t = new testClas
```

Alternative 3: BinaryFormatter (Unsafe!)

<https://docs.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide>

Serialize:

1. MemoryStream stream = `new` MemoryStream();
2. BinaryFormatter formatter = `new` BinaryFormatter();
3. `try`
4. `{`
5. formatter.Serialize(stream, objectToSerialize);

Deserialize:

1. SomeObject objectThatWasDeserialized;
2. MemoryStream stream = `new` MemoryStream();
3. stream.Write(objectAsBytes, 0, objectAsBytes.Length);
4. `stream.Seek(0, SeekOrigin.Begin);`
5. BinaryFormatter formatter = `new` BinaryFormatter();

From the lab sessions

Object Serialization: Warning



Careful!

From the lab sessions

Don't try to serialize directly GameObjects and classes derived from MonoBehaviour

Best case:

- Error when serializing

Worst case:

- No error, but the data is not consistent with your scene/project, and “polterbugs” will appear

Recommendation: Always make your own class to send over the network

(OR, for the brave ones, make an ISerializable interface)

<https://docs.microsoft.com/en-us/dotnet/api/system.runtime.serialization.iserializable?view=net-5.0>

- The idea is to use `OutputStreams` to buffer whatever data that needs to be sent by the game
 - The inverse process is considered with `InputStreams` and whatever data is received from the remote player
- The “Write” (Read) method of that stream should be used to insert (remove) the attributes of our class and provide enough buffer space for it to happen
- In particular, our game should have methods that use such “Write” (Read) method to concatenate the different attributes of the each class, ready to be then “sent” (used).

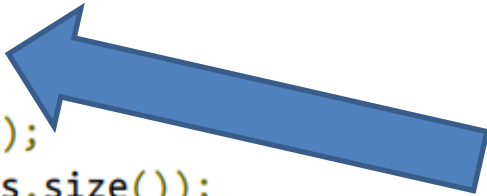
Streams (example to send)



```
// Serialize a spaceship
void Spaceship::Write(OutputStream &stream) const {
    stream.Write(mHealth);
    stream.Write(mPower);
    mHomeBase->Write(stream);
    stream.Write(mLaserShots.size());
    for (auto laserShot : mLaserShots) {
        laserShot.Write(stream);
    }
}

void Write(uint16_t val) {
    Write(&val, sizeof(val)); // Using the generic Write function
}

// Send a spaceship
void SendSpaceship(SOCKET socket, const Spaceship *spaceship) {
    OutputStream stream;
    spaceship->Write(stream);
    send(socket, stream.GetBufferPtr(), stream.GetSize(), 0);
}
```

A blue arrow points from the `Write` function definition on the right towards the `stream.Write` calls within the `Spaceship::Write` function on the left, indicating that the `Write` function is used to serialize the data into the stream.

We need ways to treat indirections and containers

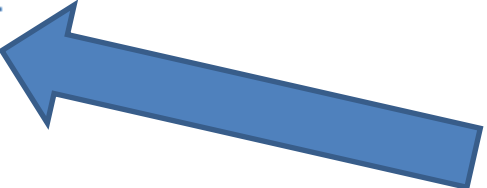
- **Inlining or embedding:** a `vector<int32_t>` cannot be just directly copied as it may contain pointers to avoid. Instead, we just serialize the size of the vector and then copy the elements in order. We will need specialized Read and Write functions capable of handling these.
- **Linking:** pointers may refer to game's objects. Due to repication, these pointers have different values in different machines, but refer to the same object. We need to send the object ID, not the pointer (or the value of the pointer).

Referenced Data (example to send)



CITM

```
// Serialize a spaceship
void Spaceship::Write(OutputStream &stream) const {
    stream.Write(mHealth);
    stream.Write(mPower);
    mHomeBase->Write(stream);
    stream.Write(mLaserShots.size());
    for (auto laserShot : mLaserShots) {
        laserShot.Write(stream);
    }
}
```

A large blue arrow pointing from the right towards the loop in the Spaceship::Write function, specifically highlighting the laserShot loop.

```
// Send a spaceship
void SendSpaceship(SOCKET socket, const Spaceship *spaceship) {
    OutputStream stream;
    spaceship->Write(stream);
    send(socket, stream.GetBufferPtr(), stream.GetSize(), 0);
}
```


- Endian compatibility is irrelevant in single-player, single-platform cases, but becomes important for cross-platform games.
- Basically, we decide on the endianness of the stream (generally the same than the origin platform), and check whether it matches the destination endianness with known methods:
 - If so, just use the data stream to read the data
 - If not, use byte swapping algorithms to deliver the data in the correct endianness

See Chapter 4 of the “Multiplayer Game Programming” book by Glazer/Madhav for:

- Way more details on all these aspects
- Specific examples for all of them
- Advice on maintainability of code



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

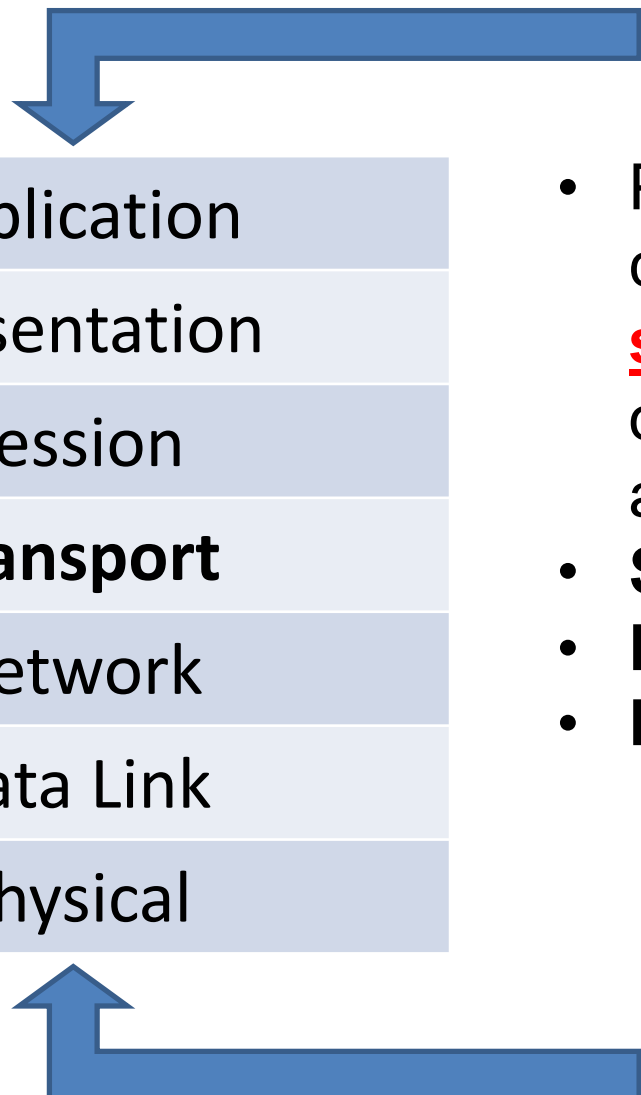
The Transport Layer

TCP and UDP

Lab Session 2: TCP / UDP

Transport Layer

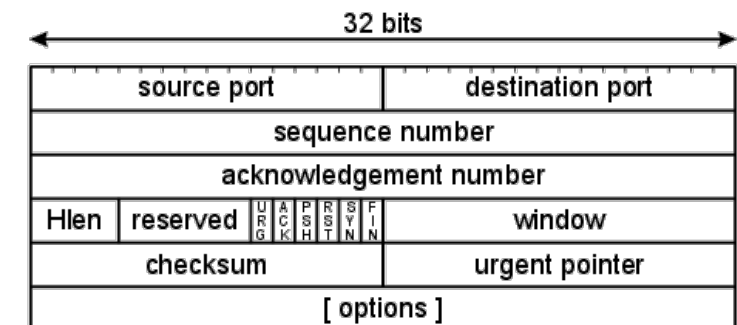
The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols to provide end to end connectivity and guarantees through sockets. Fragmentation and re assembly of messages, ordering. Also, congestion avoidance in end to end.
- **Scope:** two network end-points
- **Data unit:** datagram
- **Example:** TCP, UDP

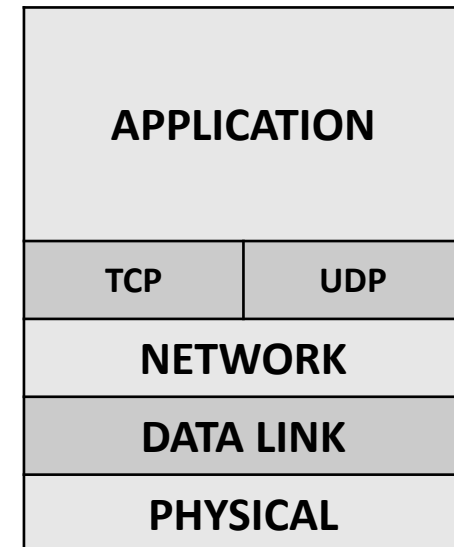
TCP header format



The wire

Transport Layer

- The transport layer allows remote processes to communicate as if they were within the same computer
- To multiplex different applications for a single IP address (single computer) the transport layer uses **ports**, like:
 - Port 25: SMTP
 - Port 80: HTTP
 - Port 110: POP3
- In the Internet there are two main transport level protocols: **UDP and TCP**
- Transport connections are made with **sockets**



Transport Layer: Sockets

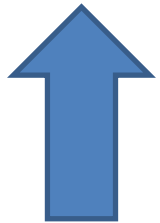
Sockets

Sockets are endpoints in the communication between two processes in the same or different machines.

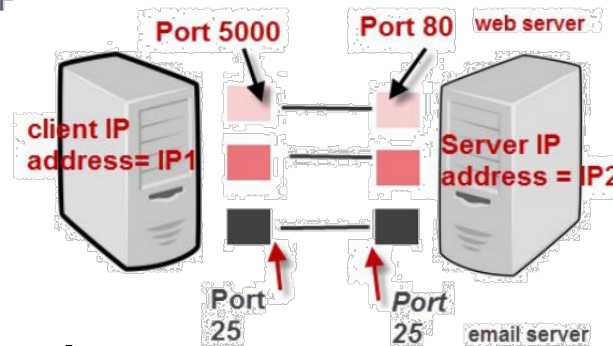
They use the same actions as any other I/O functions: Read and write

Two main types for this course: TCP and UDP

Need an Address and a Port



And send it to a destination



They take your serialized data
(in the OutputStream)

...and vice versa!
(anything received from that port
is left in an InputStream)

“Sockets are like portals connecting
our game’s process with the other player’s process,
to exchange game’s objects in the form of serialized data.”

Transport Layer: Sockets



Sockets

Sockets are endpoints in the communication between two processes in the same or different machines.

They use the same actions as any other I/O functions: Read and write

Two main types for this course: TCP and UDP

Need an Address and a Port

From the lab sessions

Sockets

To create a UDP Socket:

```
newSocket = new Socket(AddressFamily.InterNetwork,  
                        SocketType.Dgram,  
                        ProtocolType.Udp);
```

To create a TCP Socket:

```
newSocket = new Socket(AddressFamily.InterNetwork,  
                        SocketType.Stream,  
                        ProtocolType.Tcp);
```

InterNetwork -> IPv4 family of addresses
InterNetworkV6 -> IPv6 family of addresses

Dgram vs. Stream -> independent packets (Datagrams) vs continuous stream of data

Dgram always for UDP.
Stream always for TCP.

newSocket.Shutdown() -> shutdown the transfer of data, but keep socket alive

newSocket.Close() -> Stop transfers and **destroy** socket.

Transport Layer: Main Protocols



CITM

TCP vs. UDP

From the lab sessions

TCP ensures connection and messages are consistent at all times

TCP reads the messages in order of arrival (blocking)

UDP eliminates previous messages from the buffer when a new one arrives

UDP doesn't use connections neither handshakes (less safe)

UDP typically used when you need the most updated data available

TCP typically used when you need reliable communication between host and client

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - Congestion control
 - Optimal segmentation
 - Ordering

User Datagram Protocol (UDP)

- When we want to be fast and errors may be tolerated
 - Multiplex applications
 - **NO** Error detection/correction
 - **NO** Flow control
 - **NO** Congestion control
 - **NO** Optimal segmentation
 - **NO** Ordering

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - Congestion control
 - Optimal segmentation
 - Ordering

User Datagram Protocol (UDP)

- When we want to be fast and errors may be tolerated
 - Multiplex applications
 - **NO** Error detection/correction
 - **NO** Flow control
 - **NO** Congestion control
 - **NO** Optimal segmentation
 - **NO** Ordering

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - **Error detection/correction**
 - Flow control
 - Congestion control
 - Optimal segmentation
 - Ordering

Guarantees a reliable transmission from end to end. It retransmits packets that have not arrived to the destination (for whatever reason).

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - **Flow control**
 - Congestion control
 - Optimal segmentation
 - Ordering

Makes sure that the source is not sending the segments faster than the processing speed of the next node (your flow). Otherwise, segments may be lost due to buffer overflow at the intermediate nodes.

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - **Congestion control**
 - Optimal segmentation
 - Ordering

Makes sure that the source is not sending the segments faster than the speed at which the network is processing them (all flows). Otherwise, segments may be lost in due to buffer overflow at the bottleneck links. In the end, the network goes as slow as the weakest link.

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - Congestion control
 - **Optimal segmentation**
 - Ordering

Makes sure that the length of the packets avoids IP-level fragmentation as much as possible.

Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - Congestion control
 - Optimal segmentation
 - **Ordering**

Employs mechanisms to ensure the order of the packets at destination by either:

- Reordering them
(thanks to sequence numbers)
- Forcing ordered transmissions

Transport Control Protocol (TCP)



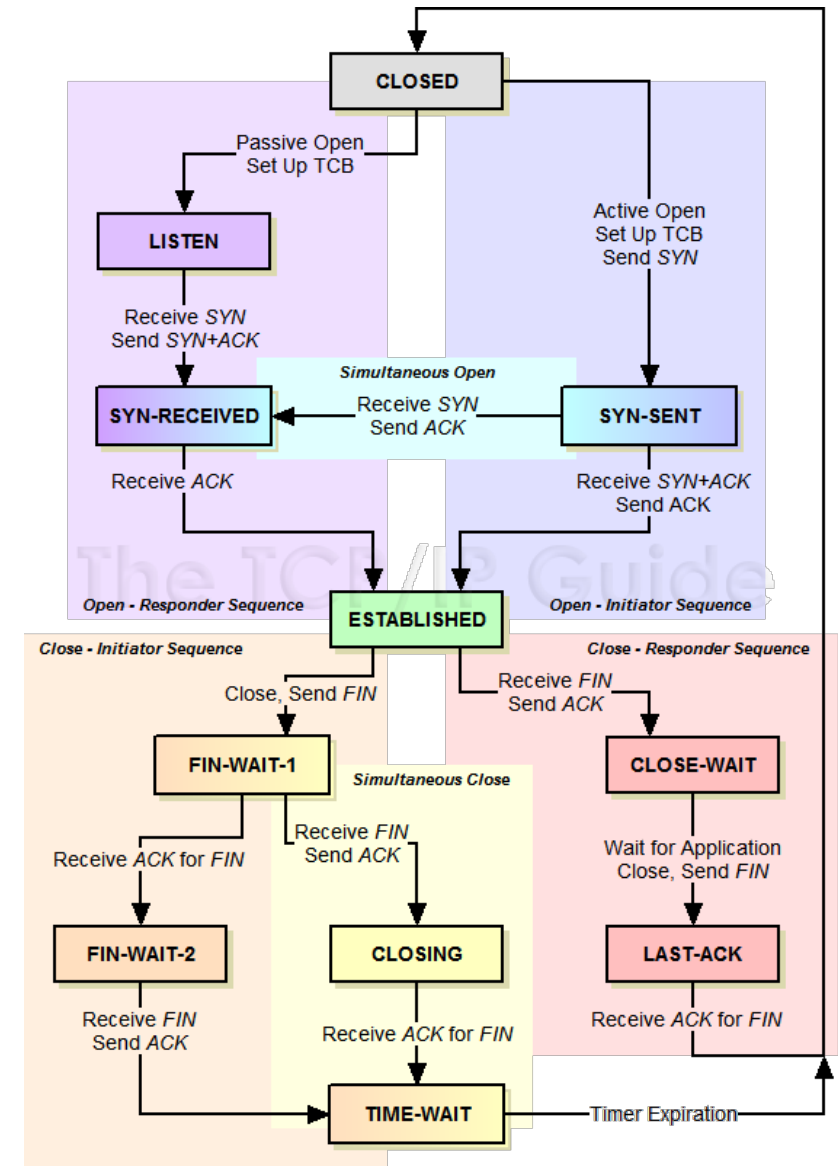
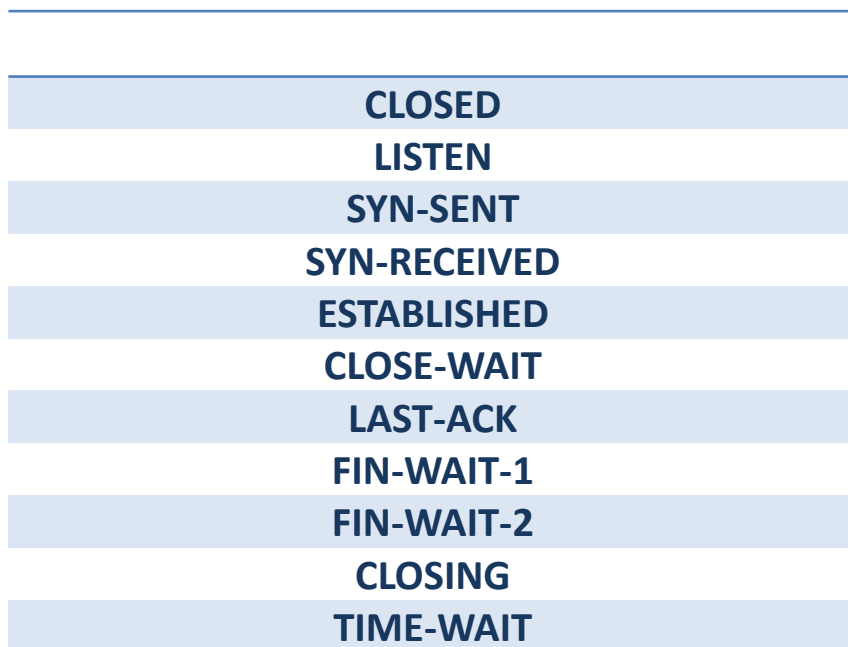
CITM

TCP is reliable because:

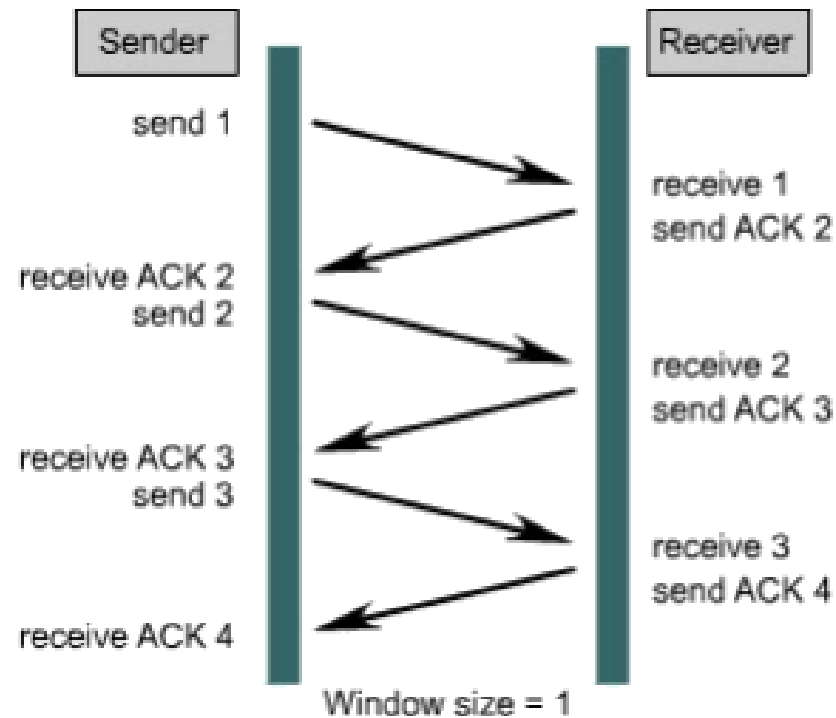
- **It is a connection-oriented protocol** as there is a handshaking mechanism to establish and close an end-to-end communication.
- **It sends end-to-end acknowledgments** whenever packets are successfully received.
- **Forces retransmissions** whenever packets are lost.
- **Uses sequence numbers** to establish packet order.
- **Uses a transmission window** to minimize packet losses.

TCP: State Diagram

- Control packets
 - SYN to initiate
 - FIN to finalize
 - ACK to confirm

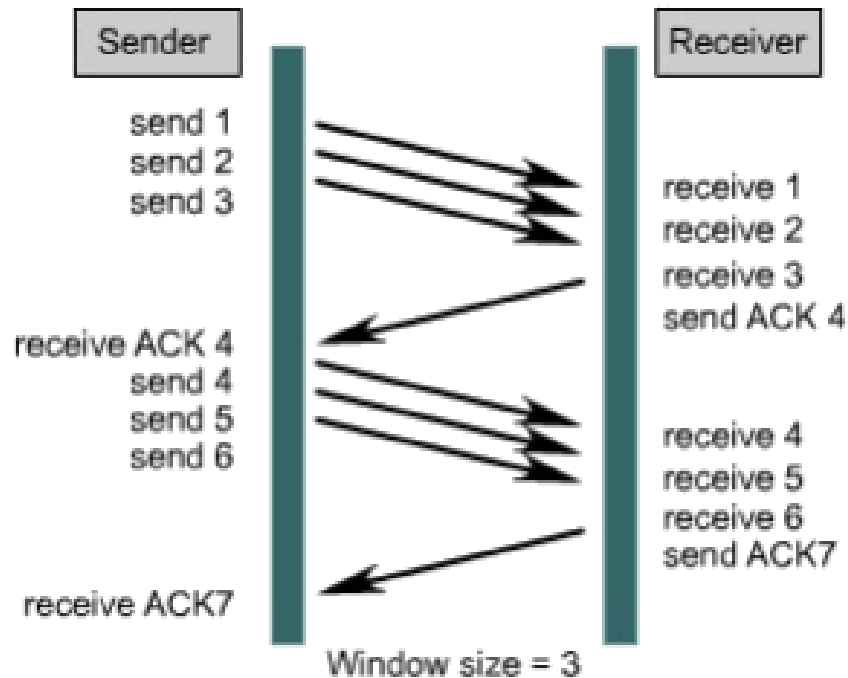


TCP: Transmission Window



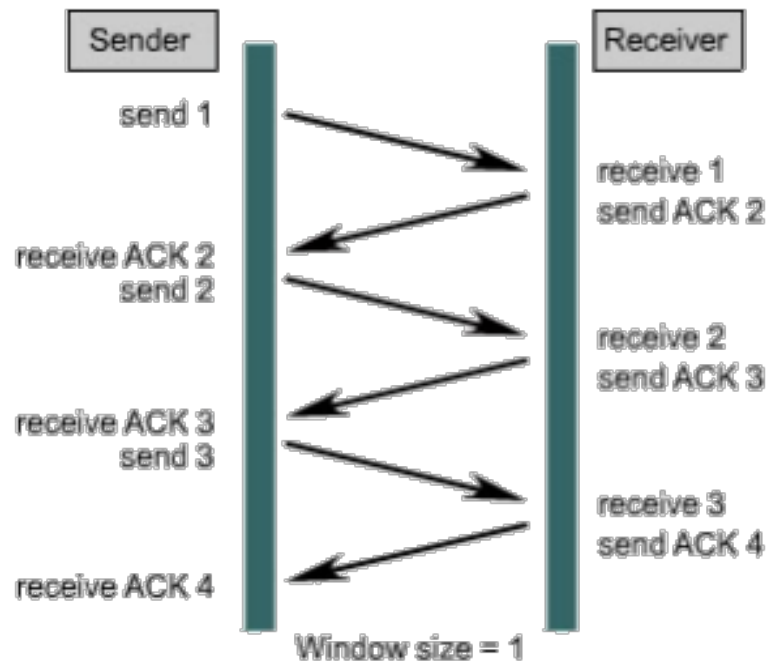
- The transmission window establishes the number of segments that can be sent without having to wait for an acknowledgment. This is used for congestion control.
 - ONE: bottleneck is slow

TCP: Transmission Window



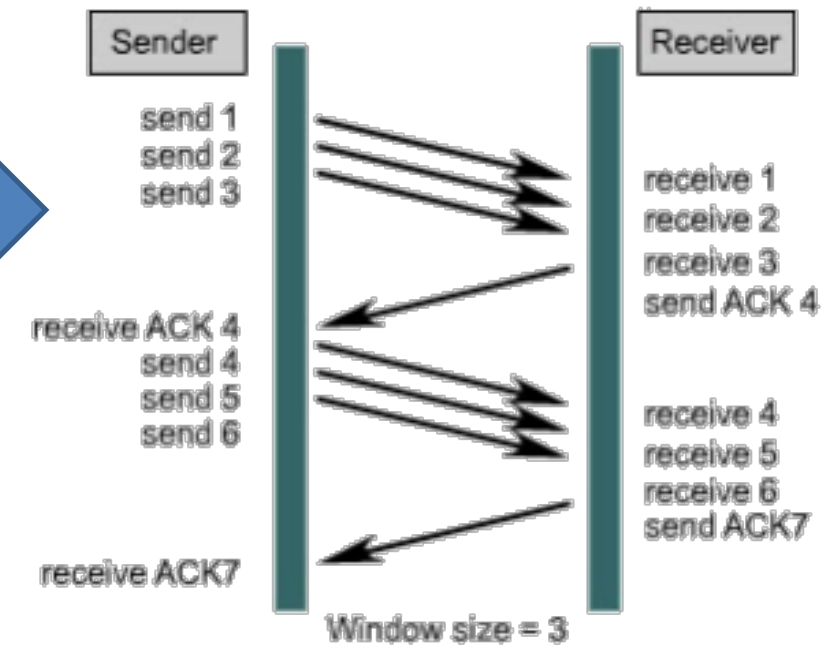
- The transmission window establishes the number of segments that can be sent without having to wait for an acknowledgment. This is used for congestion control.
 - THREE: bottleneck is relatively fast

TCP: Transmission Window Scaling

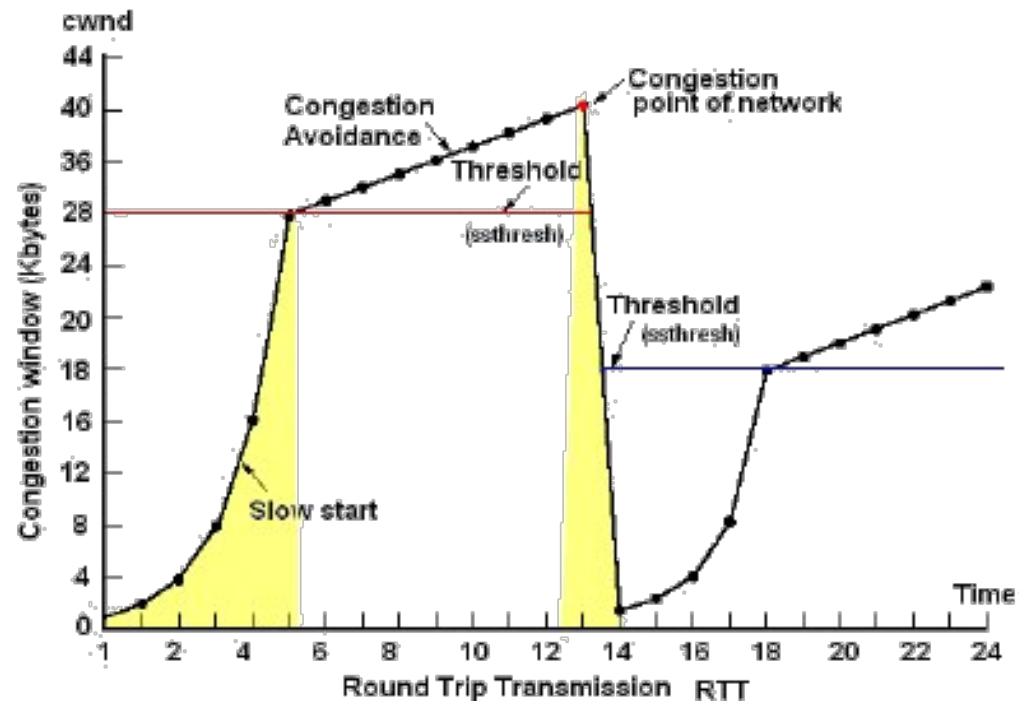


TCP window scaling

- Slow start
- Increase when OK (ACK)
- Decrease when KO (loss)

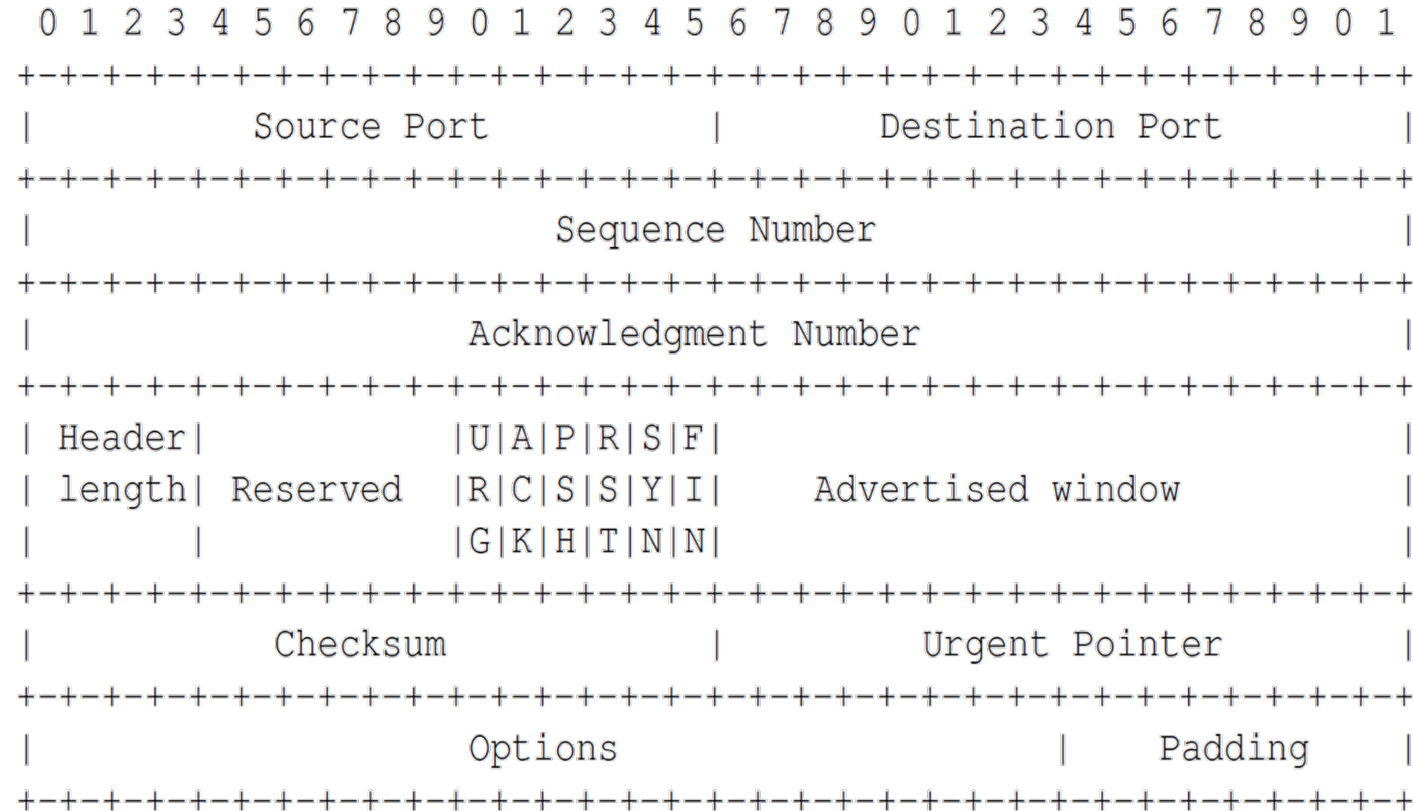


TCP: Transmission Window Scaling



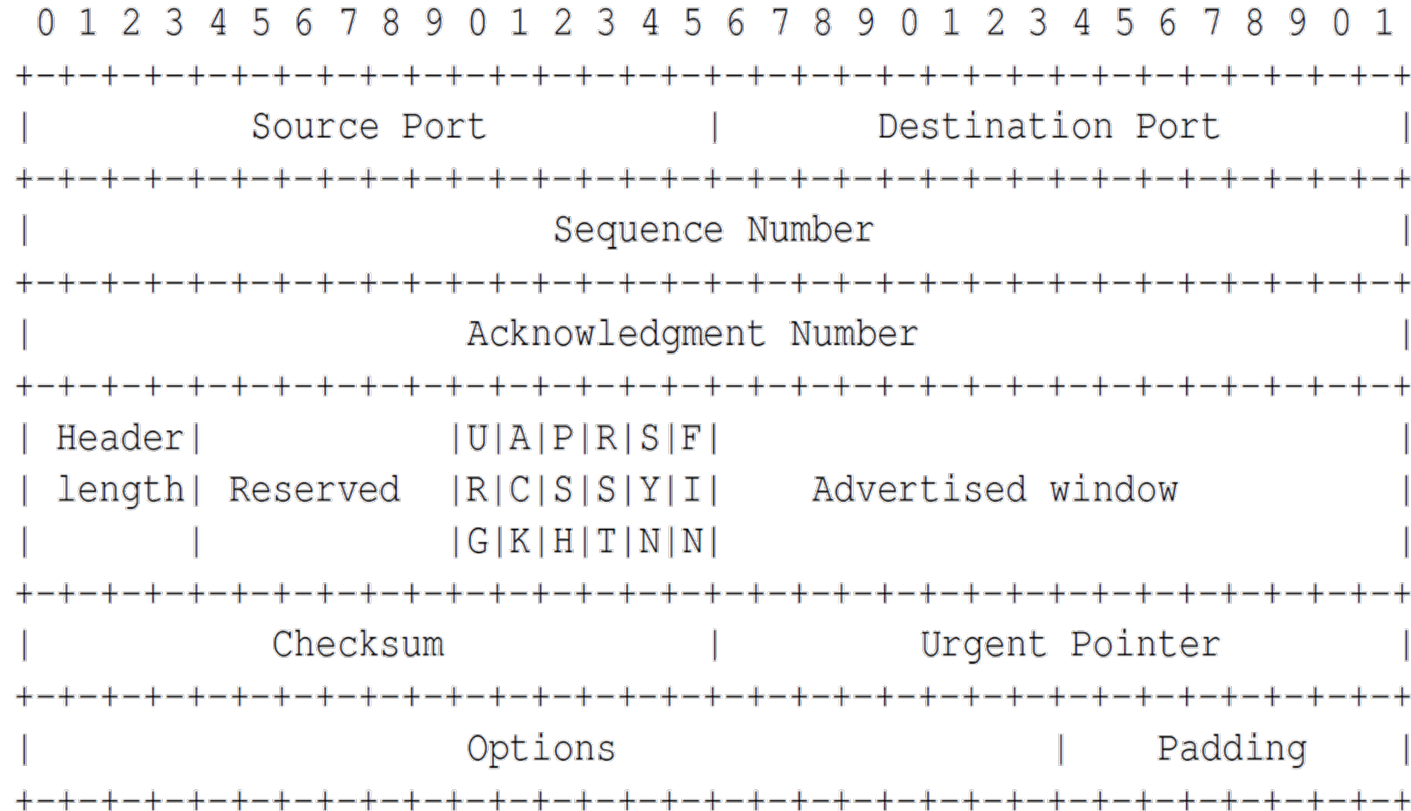
- Slow start (speedtest)
- Upon receiving ACKs we can increase the window size
- There's a limit where we need to be careful, decelerate increase
- When congestion occurs, packets will be lost and ACKs will not be sent
- The window will then be reset or decreased a lot

TCP: Packet header



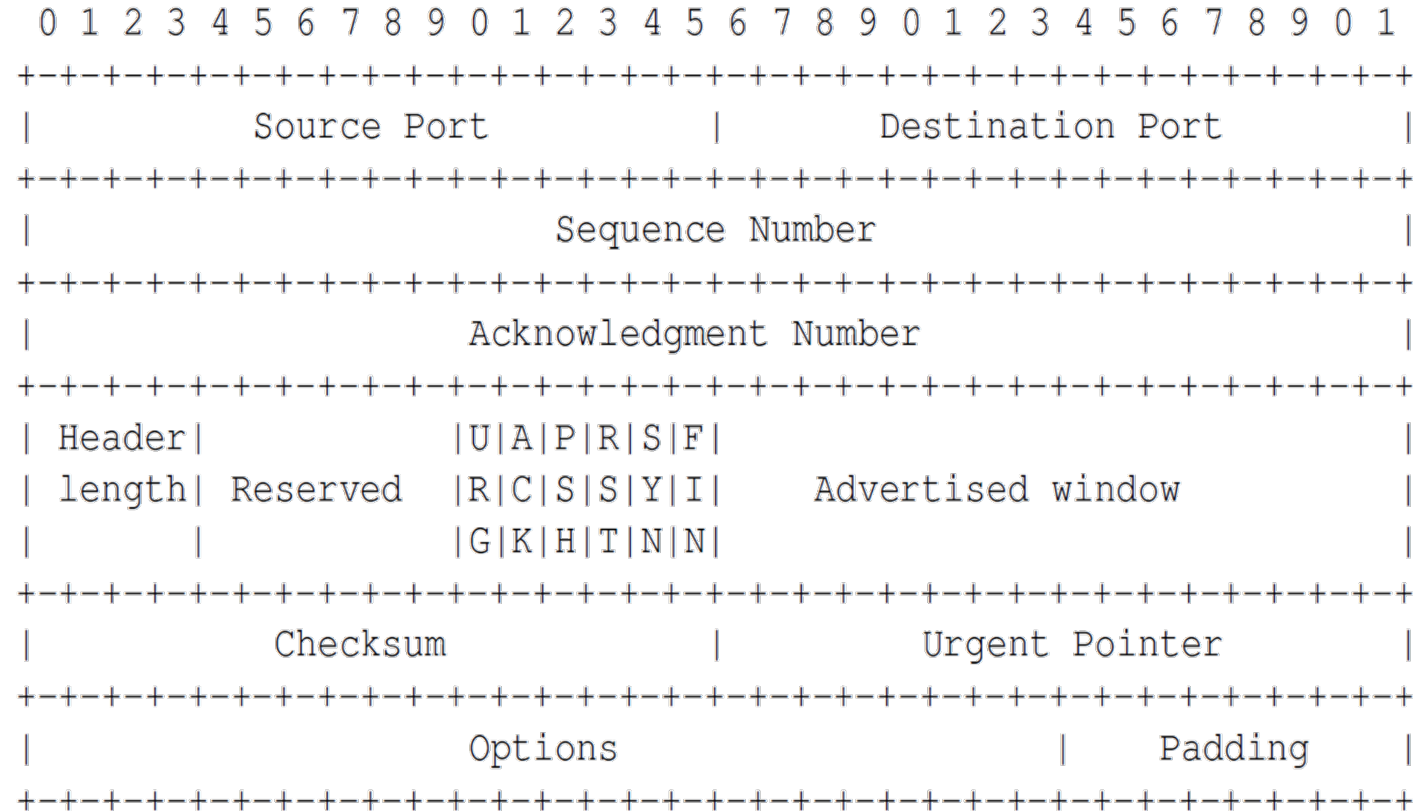
- **Source Port and Destination Port:** id of the connection used by the application.

TCP: Packet header



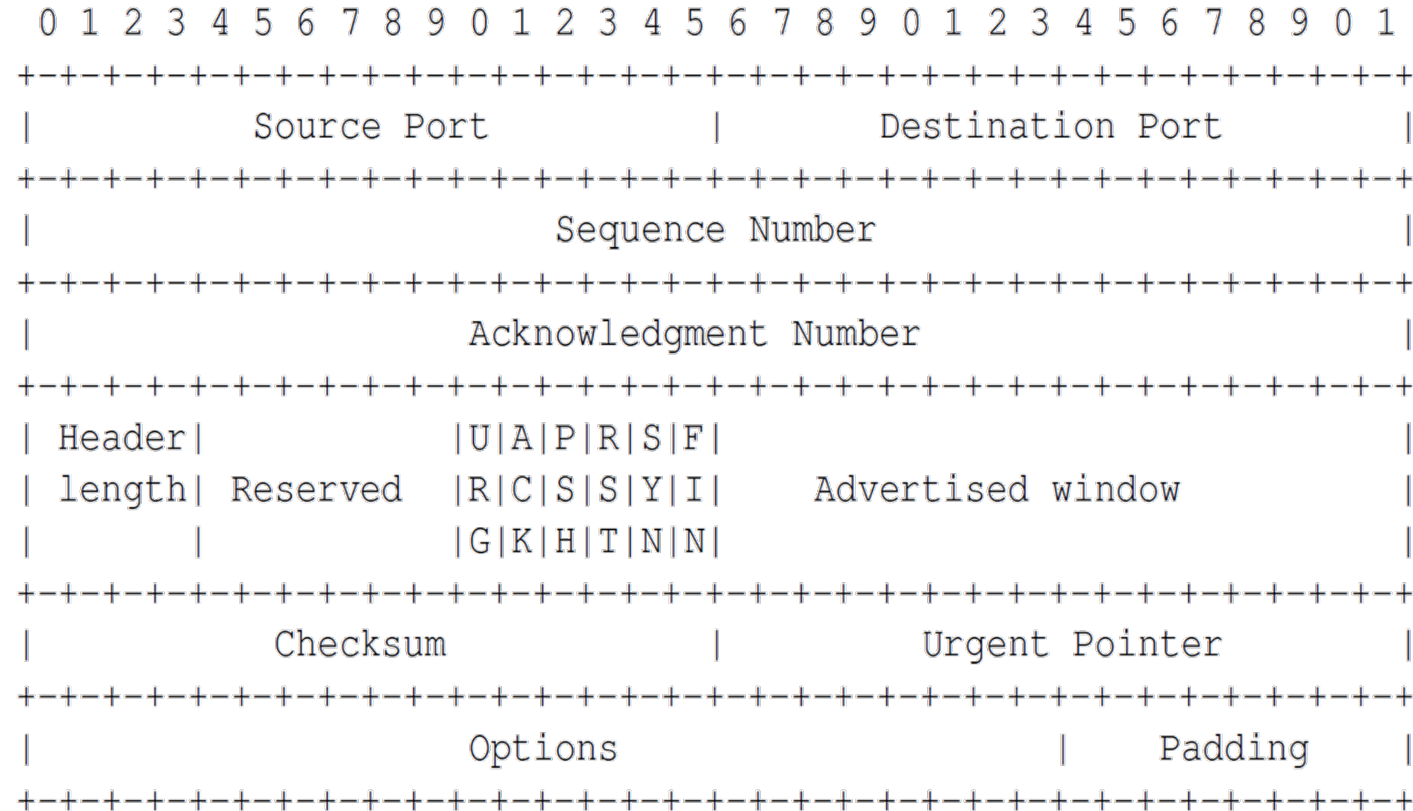
- **Sequence number:** identifies the segment (in bytes).
- **Acknowledgement number:** should match the sequence number of the segment (in bytes).

TCP: Packet header



- **Header length:** measured in 32-bit blocks (like the header length at the IP level).
- **Reserved:** bits for potential future versions of the protocol

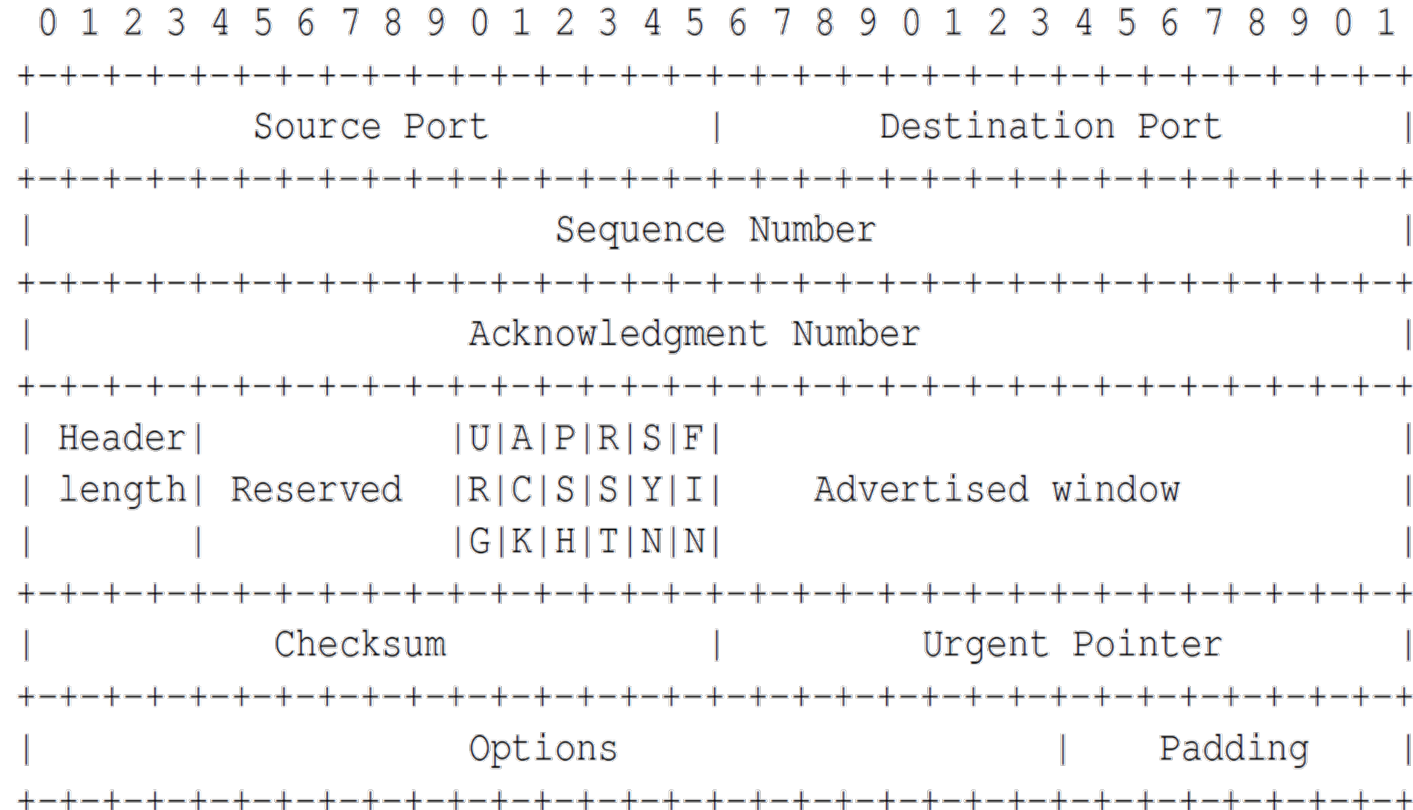
TCP: Packet header



- **FLAGS:**

URG | ACK | PSH | RST | SYN | FIN

TCP: Packet header



- **Advertised window.** Indicates next window
- **Checksum.** Mandatory (unlike in IP) because TCP is reliable
- **Padding.** Stuffing until having a 32-bits multiple

Exercise



CITM

1. Indicate the values of the field “Sequence Number” in the headers of the first three TCP segments with 256 bits each.
2. Assume a communication where each TCP segment is 256 bits long (32 bytes) and the window size is 3. Which will be the value of the “ACK number” field of the header sent back by the receiver after having successfully received six segments?



Transport Control Protocol (TCP)

- When reliable end-to-end communication is required
 - Multiplex applications
 - Error detection/correction
 - Flow control
 - Congestion control
 - Optimal segmentation
 - Ordering

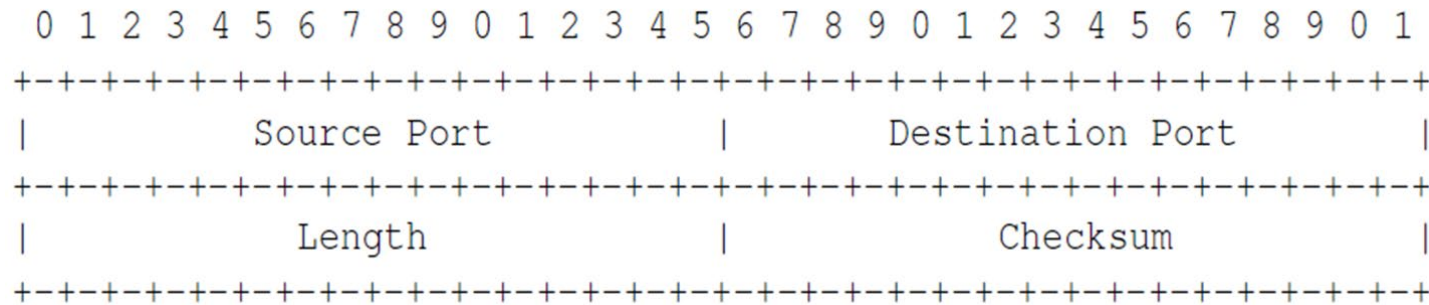
User Datagram Protocol (UDP)

- When we want to be fast and errors may be tolerated
 - Multiplex applications
 - **NO** Error detection/correction
 - **NO** Flow control
 - **NO** Congestion control
 - **NO** Optimal segmentation
 - **NO** Ordering

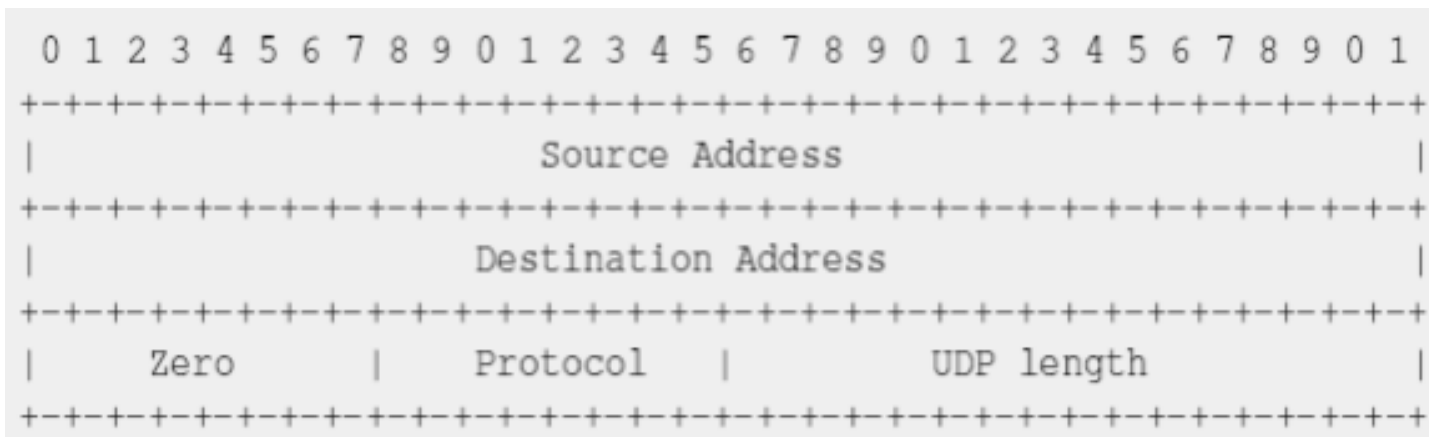
UDP is fast, not reliable:

- **Datagram-oriented** instead of connection-oriented
- **Used in “real-time” applications**, where variable delays are not tolerated well but the errors due to missing a few packets are tolerated (streaming: noise)
- **A datagram is directly discarded** if it arrives later than the instant it is supposed to be read.

UDP: Packet header



- **Source Port and Destination Port:** id of the connection used by the application.
- **Length:** of the entire datagram.
- **Checksum:** [Optional]
Includes the header and the data, adding a pseudoheader that includes addresses like if it was IPv4.



TCP or UDP?



CITM

Column Heading	TCP	UDP
Reliability	Native. Everything is delivered and processed in the order it was sent.	None. Requires custom implementation but allows fine-grained reliability.
Flow control	Will automatically slow down rate of transmission if packets are getting dropped.	None. Requires custom flow and congestion control if desired.
Memory requirements	OS must keep copies of all data sent until it is acknowledged.	Custom implementation must decide what data to keep around and what to discard immediately. Memory managed at application level.
Router prioritization	May be prioritized over UDP packets.	May be dropped before TCP packets.

Robust, Stable, Tested

Fast, “blank canvas”

Strict, not manageable

Not reliable

Example: Starsiege Tribes

- Many games use UDP and implement custom reliability
 - Virtual UDP connection
 - Acknowledgments

Virtual UDP Connection

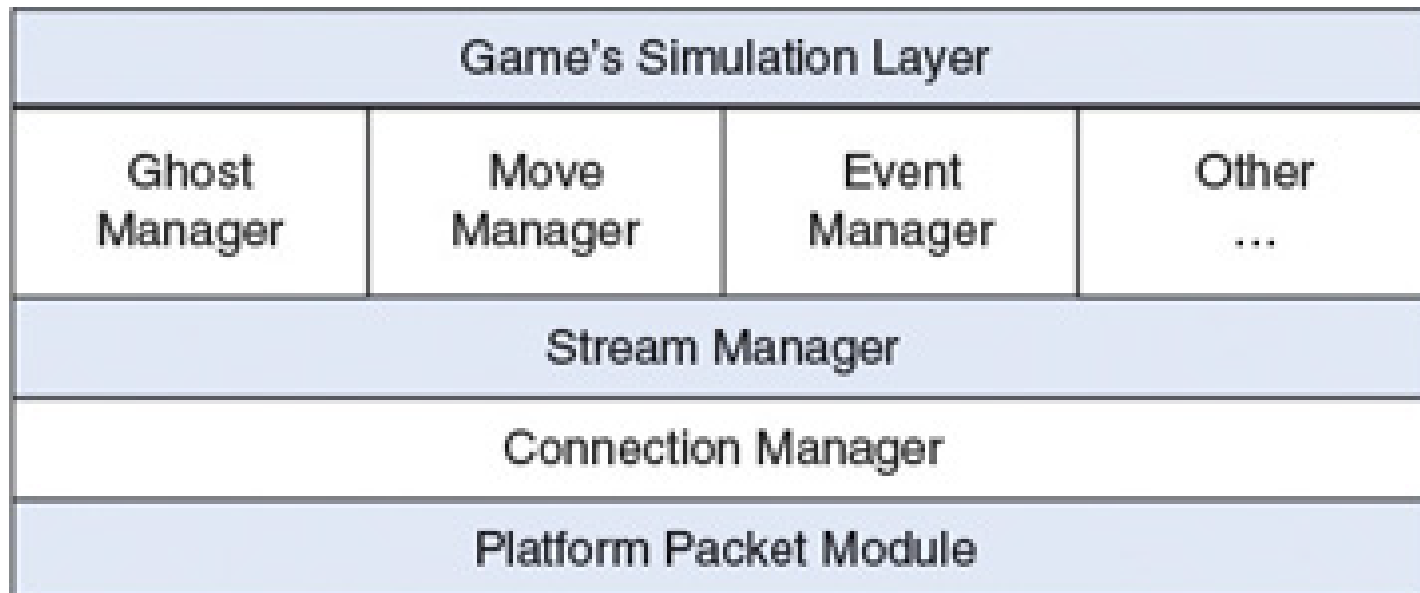
UDP is not TCP:

- Not connection oriented
- No knowledge of when the client is "alive"

How to mitigate?

- Ping packets and timeouts

From the lab
sessions



The game code being executed at FPS

Runtime managers
for different aspects

Acknowledgements

Virtual UDP connection (with pings)

Wrapper for the UDP socket



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The Protocol Stack

2.1 Introduction and upper layers

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs